



चल गुरु हो जा शुरू ओके तो ...
TXT 98.05KB

make notes out of this transcript for me , explain all these topics properly with deep concept and code explanation

MLOps Capstone Project — Complete Notes

Overview

This transcript is the **introduction video** for a capstone MLOps project. It covers:

1. What the project is about
2. Why it was delayed
3. Tools & tech stack
4. Prerequisites
5. Critical AWS/EKS billing warnings
6. Channel & career updates

1. Project Context & Motivation

Why This Project?

Throughout the MLOps playlist, individual components were taught separately:

- ML basics & pipeline building
- Data versioning (DVC)
- Experiment tracking (MLflow)
- Docker & CI/CD
- AWS services
- Kubernetes, Prometheus, Grafana, monitoring & alerting

Problem: Learners couldn't connect all the dots into one cohesive real-world project.

Solution: A capstone project that integrates **everything** end-to-end.

Previous Project vs. This Project

Aspect	Previous (Vanilla) Project	This Capstone Project
Approach	Standard Python, end-to-end	Production-grade MLOps
Data Source	MongoDB	AWS S3
Experiment Tracking	✗ Not included	✓ MLflow
Data Versioning	✗ Not included	✓ DVC
Model Registry	✗ Manual fetch	✓ Proper registry
Model Serving	Direct fetch	Registry-based serving
Model Retraining	✗	✓ Pipeline included
Deployment	Local	AWS EKS (Kubernetes)
Monitoring	Local Prometheus/Grafana	EKS-deployed + external servers
CI/CD	Basic	Modified for EKS

Note: MongoDB *can* handle structured data too — it's not only for unstructured data. But S3 was chosen as a new learning experience since it's widely used in industry.

2. Project Scope & Objectives

End-to-End Lifecycle Covered:

text

1. EXPERIMENTS (Jupyter Notebooks)

- Feature engineering, model selection, metrics

2. CODE VERSIONING (Git/GitHub)

3. DATA VERSIONING (DVC)
 - Data ingestion module with S3 as source
4. PIPELINE BUILDING + AUTOMATION (DVC)
5. EXPERIMENT TRACKING (MLflow)
6. MODEL REGISTRY (MLflow)
7. MODEL SERVING (MLflow)
8. MODEL RETRAINING PIPELINE
 - If model passes tests → goes to registry
 - Registry → serving
9. CI/CD (GitHub Actions) – modified for EKS
10. DOCKER (containerization)
11. AWS SERVICES (ECR, EKS, S3, etc.)
12. KUBERNETES DEPLOYMENT (AWS EKS)
13. MONITORING & ALERTING (Prometheus + Grafana)

ML Use Case

- **Simple text classification problem** (not fancy ML, but MLOps-heavy)
- Focus is on **MLOps depth**, not ML complexity

3. Tools & Tech Stack

Tool	Purpose
DVC	Data versioning & pipeline automation
MLflow	Experiment tracking, model registry, serving, retraining
Git & GitHub	Code versioning

Tool	Purpose
GitHub Actions	CI/CD
Docker	Containerization
AWS S3	Data source
AWS ECR	Docker image registry
AWS EKS	Managed Kubernetes (deployment)
Kubernetes	Orchestration
Prometheus	Metrics collection
Grafana	Visualization & dashboards

4. Important Career & Mindset Lessons

Key Insight: Fundamentals Are Transferable

"The fundamental knowledge is transferable."

Example given:

- Learned **GitHub** as remote repo
- First job used **Bitbucket/GitLab** instead
- Because fundamentals (clone, push, merge) were clear → no difficulty switching

Takeaway: Don't memorize tools. Understand **concepts**. Tools change; concepts don't.

Not Every Project Needs Every Component

- In real jobs, clients/employers may **not mandate** data versioning, model registry, or retraining
- Example: Current employer project doesn't need MLflow setup
- **But** if you learn it once → you can apply it anywhere

5. Prerequisites & Learning Strategy

⚠ Critical Warning

"Everything in this playlist is a **prerequisite**."

The instructor will **NOT** re-explain from scratch:

- What is data versioning?
- What is experiment tracking?
- How DVC works?

Recommended Approach When Stuck:

1. **Pause the project**
2. Go back to the relevant tutorial in the playlist
3. Re-watch that specific topic
4. Return to the project
5. Repeat as needed

This loop is **expected** and **normal**. It's not a dummy project — it's a real milestone.

6. Why the Delay? (Transparency Section)

Reasons:

1. **Full-time job** — significant time commitment
2. **Other ventures** — planning additional projects
3. **Project complexity** — most time-consuming part
4. **EKS deployment issues** — AWS account problems, long support chats (1-2 weeks just for EKS)
5. **Prometheus/Grafana setup** — kept failing inside EKS cluster
 - Solution: Set up **external servers** (AWS EC2) outside EKS cluster

- Attached them to the EKS-deployed application

Why It Was Worth It:

"This level of deployed project has **never been shown** in any YouTube course before. You guys will be the **first cohort** doing this."

7. 🚨 CRITICAL: AWS EKS Billing Warning

The Incident:

- Instructor created EKS cluster for testing
- Thought he deleted everything
- **15-20 days later**, AWS bill = ₹65,000
- Reason: **VPC, subnets, load balancers, ACLs** remained in background
- One `eksctl delete cluster` command didn't clean everything

What He Learned:

1. EKS creates **many backend resources**: EC2 instances, VPC, subnets, etc.
2. Deleting cluster $\neq$ deleting all resources
3. Must **manually verify** everything is terminated

AWS Support Experience:

- Contacted AWS chat support
- They **waived off** charges (one-time courtesy)
- Warning: "Next time, we won't waive off charges"

🛡️ Safety Protocol for Learners:

text

BEFORE doing EKS part:

- └─ Make sure you're ready
- └─ Keep cluster alive for **ONLY 1-2 days**

- └─ Delete everything properly
- └─ Verify in AWS Console:
 - | └─ EC2 instances terminated?
 - | └─ VPC deleted?
 - | └─ Subnets deleted?
 - | └─ Load balancers deleted?
 - | └─ ECR images cleaned?
- └─ Initiate AWS Chat Support
 - | └─ "Confirm all chargeable services are terminated"
- └─ WAIT for instructor's billing clarity in description box

Instructor's Promise: He will update the **description box** with exact EKS billing for 24-48 hours of usage. **Until then, avoid EKS.**

Expected Cost:

- Very basic configuration used
- 24-48 hours usage
- Likely ₹200-500 range (not more)

Psychological Warning:

"If you back out due to ₹200-300 fear, those who move forward will gain the confidence and **overtake you.**"

8. Project Division: Before EKS vs. After EKS

text

- | |
|-----------------------------------|
| BEFORE EKS (Safe for everyone) |
| └─ Experiments |
| └─ Data ingestion (S3) |
| └─ Data versioning (DVC) |
| └─ Pipeline building & automation |
| └─ Experiment tracking (MLflow) |
| └─ Model registry |
| └─ Model serving |
| └─ Model retraining |
| └─ CI/CD (GitHub Actions) |
| └─ Docker |

AFTER EKS (Proceed with caution)
├─ EKS cluster setup
├─ Kubernetes deployment
├─ Prometheus/Grafana on EKS
└─ Monitoring & Alerting

If vulnerable financially → **skip EKS part**. But you'll miss Prometheus/Grafana on EKS. However, local setup tutorials are already available.

9. Channel & Career Updates

Content Plans:

- **Tutorials** will continue (free on YouTube)
- **Gen AI / Agentic AI** content coming soon
- **Azure & GCP** services may be covered
- **Career Guidance videos** (2-3 per month):
 - Updated Data Science roadmap
 - ML learning strategy & mindset
 - DS → MLOps transition
 - Salary expectations
 - Non-tech background transitions
 - CV hacks, interview mindset, project ideas

Paid Courses:

- Instructor is **collaborating with "Fra"** (famous tech YouTuber)
- Creating **in-depth paid courses**
- First course: **ML** (releasing next month)
- Future: **Portfolio building course** (8-10 MLOps projects), **Gen AI/LLM courses**
- **Why paid?** Team effort = faster completion (2-3 months vs. 6-8 months)
- **YouTube content will ALWAYS remain free**

1-on-1 Calls:

- **Old bonus expired:** Vehicle Insurance project bonus (free 1-on-1 call) — **now ended**
- Reason: 20+ people claimed it, too many daily requests
- **New paid service:** 1-on-1 consultations restarted (link in description)
- **Free alternatives:**
 - Comment your concerns → covered in career guidance videos
 - **Webinars** with Fra (1-2 per month) → announced in Community Posts

10. Project Difficulty & Motivation

The 5% Rule:

text

100% people watch the intro video

↓

5% people reach the end of the project

↓

5% of those get CV shortlisted

↓

5% of those get interview calls

↓

5% of those get selected / transition successfully

If you complete this end-to-end, you're already in the top 5%.

Instructor's Proof:

- Built the project **3 times**:
 - Project 1 & 2: Failed at various stages
 - Project 3: Successfully replicated end-to-end
- **2 weeks** of dedicated work on Project 3 alone
- All problems pre-solved for learners
- **But** learners will still face difficulties (especially EKS, Kubeflow config)

"Most of you will give up during EKS/Kubernetes configuration. But the 5% who push through will succeed."

11. Key Concepts Explained (Deep Dive)

◆ What is MLOps?

MLOps = **Machine Learning + Operations**

- Bridge between ML models and production
- Automates: data pipelines, training, deployment, monitoring
- Ensures: reproducibility, scalability, reliability

◆ What is DVC (Data Version Control)?

- Git for **data & models**
- Tracks large files without bloating Git
- Creates **DAG pipelines** (directed acyclic graphs)
- Enables **reproducibility** of experiments
- Commands: `dvc init` , `dvc add` , `dvc repro` , `dvc push`

◆ What is MLflow?

Four components:

1. **Tracking** — log params, metrics, artifacts
2. **Projects** — package code for reproducibility
3. **Models** — standard format for models
4. **Registry** — central model store with versioning

◆ What is Model Registry?

- Centralized hub for ML models
- Version control for models
- Stage transitions: `Staging` → `Production` → `Archived`

- Enables **model serving** directly from registry

◆ What is Model Serving?

- Deploying model as an API/service
- MLflow: `mlflow models serve -m models:/model_name/production`
- Scalable, versioned, monitored

◆ What is Model Retraining?

- Automated pipeline to retrain on new data
- Triggers: schedule, data drift, performance drop
- If new model passes tests → registry → serving

◆ What is EKS?

- **Elastic Kubernetes Service** (AWS managed K8s)
- Abstracts control plane management
- Creates: EC2 nodes, VPC, subnets, load balancers
- **Expensive** if left running

◆ What is Prometheus + Grafana?

- **Prometheus**: Metrics collection & alerting
- **Grafana**: Visualization dashboards
- Together: Monitor model performance, API latency, resource usage

12. Code & Implementation Notes

Data Ingestion (S3 as source):

```
python
```

```
# Concept: Fetch data from S3 bucket
import boto3
import pandas as pd
```

```
s3 = boto3.client('s3')
obj = s3.get_object(Bucket='your-bucket', Key='data.csv')
df = pd.read_csv(obj['Body'])
```

DVC Pipeline (dvc.yaml):

yaml

```
stages:
  data_ingestion:
    cmd: python src/data_ingestion.py
    deps:
      - src/data_ingestion.py
    outs:
      - data/raw/data.csv

  data_preprocessing:
    cmd: python src/data_preprocessing.py
    deps:
      - data/raw/data.csv
      - src/data_preprocessing.py
    outs:
      - data/processed/data.csv

  model_training:
    cmd: python src/model_training.py
    deps:
      - data/processed/data.csv
      - src/model_training.py
    outs:
      - models/model.pkl
```

MLflow Tracking:

python

```
import mlflow
import mlflow.sklearn

with mlflow.start_run():
    mlflow.log_param("n_estimators", 100)
    mlflow.log_metric("accuracy", 0.95)
    mlflow.sklearn.log_model(model, "model")
```

MLflow Model Registry:

python

```
# Register model
mlflow.register_model(
    "runs:/<run_id>/model",
    "text_classifier"
)

# Transition to production
client = mlflow.tracking.MlflowClient()
client.transition_model_version_stage(
    name="text_classifier",
    version=1,
    stage="Production"
)
```

Model Serving from Registry:

bash

```
mlflow models serve \
  -m models:/text_classifier/Production \
  -p 5001
```

EKS Cluster Creation:

bash

```
eksctl create cluster \
  --name mlops-cluster \
  --region us-east-1 \
  --nodegroup-name standard-workers \
  --node-type t3.medium \
  --nodes 2
```

EKS Cleanup (CRITICAL):

bash

```
# Delete cluster
eksctl delete cluster --name mlops-cluster --region us-east-1
```

```
# MANUALLY VERIFY in AWS Console:  
# - EC2 instances terminated  
# - VPC deleted  
# - Subnets deleted  
# - Load balancers deleted  
# - ECR images cleaned  
  
# Then contact AWS Support to confirm
```

13. Final Takeaways

#	Takeaway
1	Fundamentals > Tools — concepts are transferable
2	Everything in playlist is prerequisite — don't skip
3	Pause & revisit when stuck — it's normal
4	EKS is expensive — delete resources carefully
5	Wait for instructor's billing clarity before EKS
6	5% rule — completing this puts you in top 5%
7	YouTube content stays free — paid courses for faster learning
8	Build 3 times — failure is part of the process
9	Portfolio value — this project will stand out
10	Consistency wins — 5-6 videos, end-to-end replication

14. Next Steps

1. ☒ Subscribe + press bell icon
2. ☒ Complete previous vanilla project (if not done)

3.  Revise: DVC, MLflow, Docker, CI/CD, Kubernetes
4.  Wait for EKS billing clarity in description
5.  Follow along: 5-6 videos, end-to-end
6.  Join webinars / community posts for guidance
7.  Build, fail, rebuild — until it works

Instructor's Final Words:

"You will face difficulties. I can guarantee this. But if you complete this end-to-end, you're in the top 5%. And that 5% gets shortlisted, interviewed, and hired. So push through."

End of Notes



ओके सो लेट्स गेट स्टार्टेड सो ...
TXT 147.08KB

make notes out of this transcript for me , explain all these topics properly with deep concept and code explanation

MLOps Capstone Project — Notes: Part 2 (Project Setup & Experiment Tracking)

Scope of This Video

This lecture covers the **first practical step** of any MLOps project:

1. **Project structure setup** using Cookiecutter
2. **Connecting to GitHub** (init, commit, push)
3. **DagsHub + MLflow setup** for remote experiment tracking
4. **Performing experiments** on different notebooks/scripts
5. **Comparing experiments** to pick the best strategy for end-to-end development

1. Cookiecutter — Project Structure Tool

What is Cookiecutter?

- A tool that **generates a project skeleton/template**
- Alternative to manually writing a `template.py` script (used in previous project)
- Creates standardized folder structure for ML projects

Installation & Usage

```
bash

pip install cookiecutter

cookiecutter -c v1 https://github.com/drivendata/cookiecutter-data-science
```

Prompts you'll be asked:

Prompt	Example
project_name	capstone_project
repo_name	capstone_project
author_name	Your name
description	End-to-end MLOps project
license	(optional)
s3_bucket	(leave blank, add later)
aws_profile	(leave blank)
python_version	3.10

Generated Structure

```
text
```

```

capstone_project/
├─ data/
├─ docs/
├─ models/
├─ notebooks/
├─ references/
├─ reports/
├─ src/
│   └─ models/      ← rename to avoid conflict
├─ .gitignore
├─ LICENSE
├─ Makefile
├─ README.md
├─ requirements.txt
└─ setup.py

```

⚠ Important Fix

Inside `src/`, there's a **models** folder that conflicts with the **outer models** folder.

```
bash
```

```
# Rename src/models → src/model
```

This avoids confusion when writing:

- `model_training.py`
- `model_evaluation.py`

Move Files to Root

After generation, the structure is nested:

```

text

ml_series/
└─ capstone_project/
    └─ capstone_project/  ← actual project here

```

Fix: Cut all contents from inner folder → paste to outer → delete empty folder.

2. Virtual Environment Setup

```
bash
```

```
conda create -n atlas python=3.10 -y  
conda activate atlas
```

Why Python 3.10? Avoid dependency conflicts with the instructor's environment.

3. Git & GitHub Setup

```
bash
```

```
git init  
git add .  
git commit -m "project structure ready"  
git remote add origin <your-repo-url>  
git push origin main
```

Dummy Files to Add

- dummy.txt (placeholder)
- .gitignore (Python)
- LICENSE (optional)

Instructor's Note: Repo is kept **public** so learners can access the code. But **AWS/EKS** parts should **NOT** be rushed — wait for the tutorial.

4. DagsHub + MLflow — Remote Experiment Tracking

Why DagsHub?

- Free remote MLflow tracking server
- Integrates with GitHub, GitLab, Bitbucket
- Provides UI for experiments, models, data

Setup Steps

1. Sign in at dagshub.com
2. Click "Create New Repo" → "Connect a Repository"
3. Choose **GitHub** → authorize → select your repo
4. Copy the **Tracking URI** and **Credentials**

DagHub Credentials Setup

python

```
# mlflow_tracking.py
import os
import mlflow

os.environ["MLFLOW_TRACKING_URI"] = "https://dagshub.com/<user>/<repo>.mlflow"
os.environ["MLFLOW_TRACKING_USERNAME"] = "<username>"
os.environ["MLFLOW_TRACKING_PASSWORD"] = "<token>"
```

⚠ **Security:** Store credentials in `credentials.txt` → add to `.gitignore`

gitignore

credentials.txt

Install Required Packages

bash

```
pip install dagshub mlflow
```

5. The ML Problem: IMDB Sentiment Classification

Problem Statement

- **Input:** Movie review text
- **Output:** Positive (1) or Negative (0) sentiment

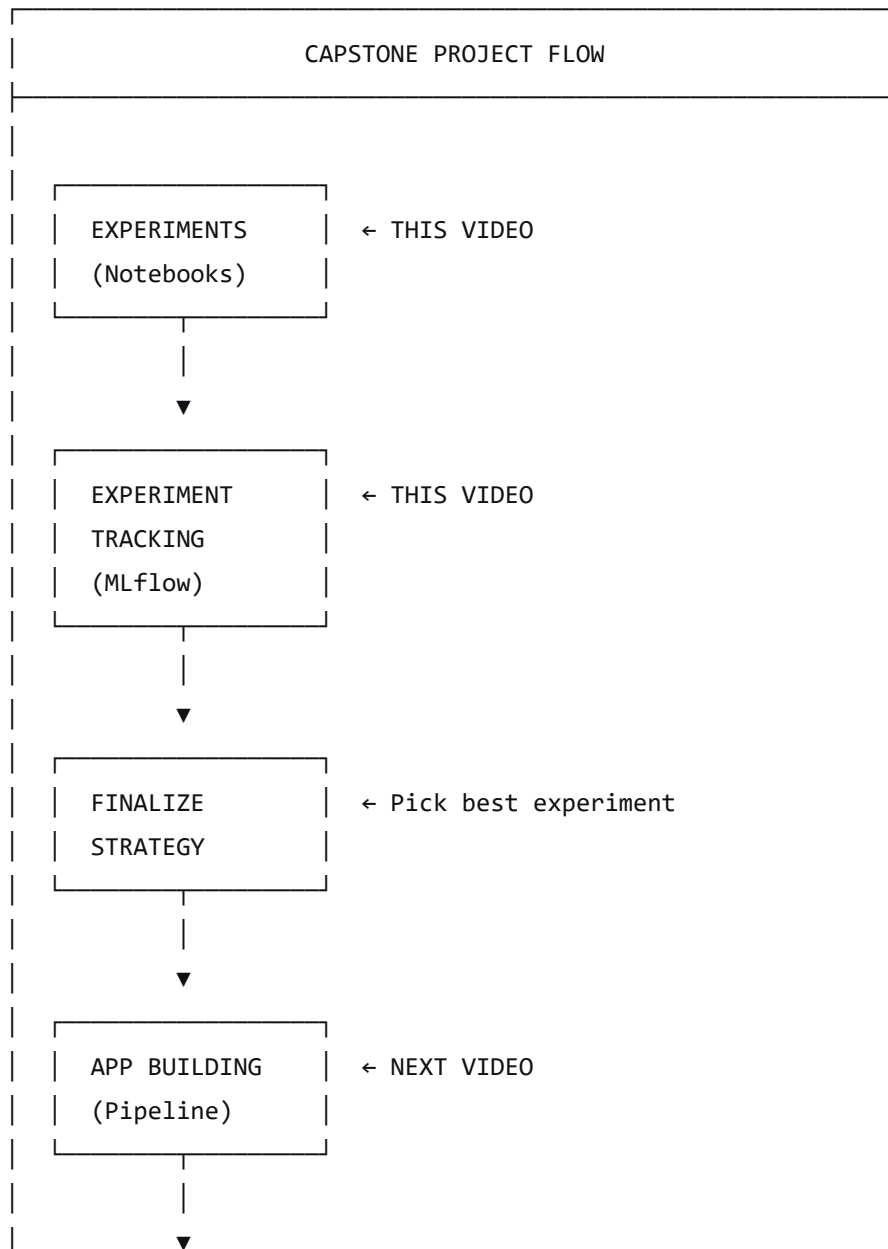
- **Dataset:** IMDB (50,000+ reviews)

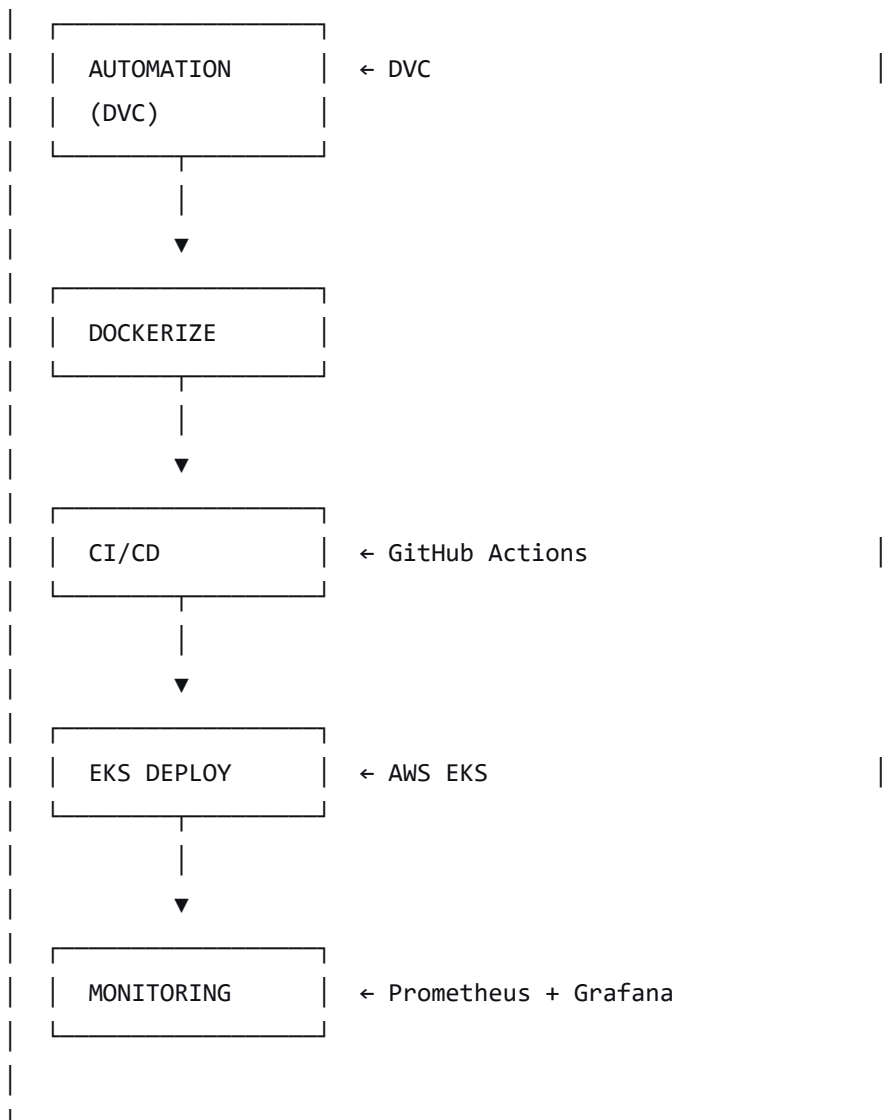
Why a Subset?

- Full dataset = slow processing (feature engineering, cleaning, preprocessing)
- **Solution:** 500 positive + 500 negative = **1000 rows** (`data.csv`)
- **Balanced dataset** → prevents model bias
- Once project is complete → replace with full IMDB dataset

6. Project Architecture (360° View)

text





Key Insight: You NEVER jump directly into app development. You FIRST experiment, THEN build.

7. AWS EKS Billing Estimate (Real Data)

Detail	Value
Usage Duration	~1-1.5 days
Services Used	EKS cluster + Prometheus + Grafana
Estimated Cost	Under \$6 (~₹500)
Forecasted (if left running)	Higher (AWS assumes continued usage)

Instructor's Advice:

- If comfortable with ~₹500 → do the AWS part
- If not → watch tutorials, skip execution for now
- **Wait for EKS billing clarity in description box** before executing
- 1 day is **more than enough** if no exceptional problems (3–4 hours realistically)

AWS Budget Setup (Coming Later)

Will be shown during deployment phase.

8. Experiment 1: Baseline Logistic Regression

Notebook: 1_experiment_baseline.ipynb

Data Loading & Subset Creation

python

```
import pandas as pd

# Load full IMDB dataset
df = pd.read_csv("IMDB Dataset.csv")

# Take balanced subset (500 pos + 500 neg)
df_positive = df[df['sentiment'] == 'positive'].sample(500)
df_negative = df[df['sentiment'] == 'negative'].sample(500)

# Combine and save
data = pd.concat([df_positive, df_negative])
data.to_csv("data.csv", index=False)
```

Text Cleaning Functions

python

```
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
```

```
nltk.download('stopwords')
nltk.download('wordnet')

lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words('english'))

def remove_urls(text):
    return re.sub(r'http\S+|www\S+|https\S+', '', text)

def remove_emails(text):
    return re.sub(r'\S+@\S+', '', text)

def remove_punctuation(text):
    return re.sub(r'[\W\s]', '', text)

def remove_numbers(text):
    return re.sub(r'\d+', '', text)

def lowercase(text):
    return text.lower()

def remove_stopwords(text):
    return ' '.join([w for w in text.split() if w not in stop_words])

def lemmatize(text):
    return ' '.join([lemmatizer.lemmatize(w) for w in text.split()])

def clean_text(text):
    text = lowercase(text)
    text = remove_urls(text)
    text = remove_emails(text)
    text = remove_punctuation(text)
    text = remove_numbers(text)
    text = remove_stopwords(text)
    text = lemmatize(text)
    return text
```

Apply Cleaning

python

```
df['review'] = df['review'].apply(clean_text)

# Keep only positive/negative
df = df[df['sentiment'].isin(['positive', 'negative'])]
```

```
# Encode Labels
df['sentiment'] = df['sentiment'].map({'positive': 1, 'negative': 0})
```

CountVectorizer (Bag of Words)

python

```
from sklearn.feature_extraction.text import CountVectorizer

vectorizer = CountVectorizer(max_features=50)
X = vectorizer.fit_transform(df['review']).toarray()
y = df['sentiment']
```

Concept: Bag of Words

- Takes top N frequent words as columns
- Each row = word frequency vector
- Example: "good" → column good = 1 if present, else 0

Train-Test Split

python

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

MLflow Remote Tracking Setup

python

```
import mlflow
import mlflow.sklearn
import os

os.environ["MLFLOW_TRACKING_URI"] = "https://dagshub.com/<user>/<repo>.mlflow"
os.environ["MLFLOW_TRACKING_USERNAME"] = "<username>"
os.environ["MLFLOW_TRACKING_PASSWORD"] = "<token>"
```

```
mlflow.set_experiment("Logistic Regression Baseline")
```

Full Training Loop with MLflow

```
python
```

```
import time
import logging
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

logging.basicConfig(level=logging.INFO)

start_time = time.time()

with mlflow.start_run():
    logging.info("Starting MLflow run")

    # Log preprocessing parameters
    mlflow.log_param("vectorizer", "Bag of Words")
    mlflow.log_param("max_features", 50)
    mlflow.log_param("test_size", 0.2)

    # Initialize and train model
    model = LogisticRegression()
    logging.info("Training Logistic Regression model")
    model.fit(X_train, y_train)

    # Log model parameters
    mlflow.log_param("C", model.C)
    mlflow.log_param("solver", model.solver)

    # Predictions
    y_pred = model.predict(X_test)

    # Metrics
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    mlflow.log_metric("accuracy", accuracy)
    mlflow.log_metric("precision", precision)
    mlflow.log_metric("recall", recall)
    mlflow.log_metric("f1_score", f1)
```

```
logging.info(f"Accuracy: {accuracy}, Precision: {precision}, Recall: {recall}, F1: {f1}")

# Log model
mlflow.sklearn.log_model(model, "model")

end_time = time.time()
logging.info(f"Model training and logging completed in {end_time - start_time:.2f} seconds")
```

Experiment Comparison (Run 1 vs Run 2)

Parameter	Run 1	Run 2
max_features	50	100
test_size	0.2	0.25
Accuracy	Higher	Lower

Insight: More features $\neq$ better. Overfitting risk increases.

9. Experiment 2: Multiple Algorithms + Vectorizers

Notebook: 2_experiments_bow_vs_tfidf.ipynb

Config Dictionary Approach

python

```
config = {
    "data_path": "data.csv",
    "test_size": 0.2,
    "mlflow_tracking_uri": "https://dagshub.com/<user>/<repo>.mlflow",
    "experiment_name": "Bag of Words vs TF-IDF",
    "repo_owner": "<user>",
    "repo_name": "<repo>"
}
```

Vectorizers

python

```
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer

vectorizers = {
    "Bag of Words": CountVectorizer(max_features=50),
    "TF-IDF": TfidfVectorizer(max_features=50)
}
```

Algorithms

python

```
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier

algorithms = {
    "Logistic Regression": LogisticRegression(),
    "Multinomial Naive Bayes": MultinomialNB(),
    "Random Forest": RandomForestClassifier(),
    "Gradient Boosting": GradientBoostingClassifier()
}
```

Nested Runs with Grid Search

python

```
def log_model_params(algorithm_name, model):
    """Log algorithm-specific parameters."""
    if algorithm_name == "Logistic Regression":
        mlflow.log_param("C", model.C)
    elif algorithm_name == "Gradient Boosting":
        mlflow.log_param("n_estimators", model.n_estimators)
        mlflow.log_param("learning_rate", model.learning_rate)
    elif algorithm_name == "Random Forest":
        mlflow.log_param("n_estimators", model.n_estimators)
    # Add more as needed

def train_and_evaluate(algorithm_name, algorithm, vectorizer_name, vectorizer, X_train, X_test, y_train, y_test):
    with mlflow.start_run(run_name=f"{algorithm_name} + {vectorizer_name}", nested=True):
```

```

# Vectorize
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

# Train
model = algorithm
model.fit(X_train_vec, y_train)

# Log params
mlflow.log_param("algorithm", algorithm_name)
mlflow.log_param("vectorizer", vectorizer_name)
mlflow.log_param("test_size", 0.2)
log_model_params(algorithm_name, model)

# Predict
y_pred = model.predict(X_test_vec)

# Metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

mlflow.log_metric("accuracy", accuracy)
mlflow.log_metric("precision", precision)
mlflow.log_metric("recall", recall)
mlflow.log_metric("f1_score", f1)

# Parent run
with mlflow.start_run(run_name="Bag of Words vs TF-IDF - Parent"):
    for algo_name, algo in algorithms.items():
        for vec_name, vec in vectorizers.items():
            train_and_evaluate(algo_name, algo, vec_name, vec, X_train, X_test, y_train,
                               y_test)

```

Run Command

bash

python notebooks/2_bow_vs_tfidf.py

Common Errors & Fixes

Error 1: FileNotFoundError: data.csv

```
python
```

```
# Fix: Use relative path
data_path = "notebooks/data.csv" # or copy data.csv to root
```

Error 2: ModuleNotFoundError for local packages

```
bash
```

```
# Fix: Install local package
pip install -e .
# or
pip install -r requirements.txt
```

Error 3: requirements.txt conflict

- Cookiecutter creates its own requirements.txt
- **Fix:** Replace with your own → pip install -r requirements.txt

Experiment Results Comparison

Algorithm	Vectorizer	Accuracy
Gradient Boosting	Bag of Words	Highest
Logistic Regression	Bag of Words	High
Multinomial NB	Bag of Words	Medium
Logistic Regression	TF-IDF	Lower

Insight: Gradient Boosting + Bag of Words performed best, but instructor chose **Logistic Regression** as it's simpler to deploy and the project structure is built around it.

10. Experiment 3: Hyperparameter Tuning

Notebook: 3_1r_hyperparameter_tuning.ipynb

Objective

- Algorithm & vectorizer chosen: **Logistic Regression + Bag of Words**
- Now find **best hyperparameters** for Logistic Regression

Load & Prepare Function

python

```
def load_and_prepare_data(file_path, test_size=0.2):  
    df = pd.read_csv(file_path)  
    df['review'] = df['review'].apply(clean_text)  
    df = df[df['sentiment'].isin(['positive', 'negative'])]  
    df['sentiment'] = df['sentiment'].map({'positive': 1, 'negative': 0})  
  
    vectorizer = CountVectorizer(max_features=50)  
    X = vectorizer.fit_transform(df['review']).toarray()  
    y = df['sentiment']  
  
    return train_test_split(X, y, test_size=test_size, random_state=42)
```

Parameter Grid

python

```
param_grid = {  
    "C": [0.01, 0.1, 1, 10, 100],  
    "penalty": ["l1", "l2"],  
    "solver": ["liblinear"] # supports both l1 and l2  
}
```

Hyperparameter Tuning with MLflow

python

```
mlflow.set_experiment("Logistic Regression Hyperparameter Tuning")  
  
with mlflow.start_run(run_name="LR Hyperparameter Tuning"):  
    for C in param_grid["C"]:  
        for penalty in param_grid["penalty"]:  
            with mlflow.start_run(nested=True):  
                model = LogisticRegression(C=C, penalty=penalty, solver="liblinear")  
                model.fit(X_train, y_train)  
                y_pred = model.predict(X_test)
```

```
mlflow.log_param("C", C)
mlflow.log_param("penalty", penalty)
mlflow.log_param("solver", "liblinear")

accuracy = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

mlflow.log_metric("accuracy", accuracy)
mlflow.log_metric("f1_score", f1)
```

Best Parameters Found

text

Best Params: {'C': 10, 'penalty': 'l2', 'solver': 'liblinear'}
Best F1 Score: ~0.70

Note: Results may fluctuate on re-run due to random data shuffling.

11. DVC Initialization (Local S3 Setup)

Why DVC?

- Data versioning
- Pipeline automation
- Reproducibility

Setup

bash

```
dvc init
```

Create Local Remote (Temporary)

bash

```
mkdir local_s3  
dvc remote add -d my_local s3://local_s3 # placeholder
```

Note: Actual S3 remote will be configured later. Local folder is a skeleton.

Add to .gitignore

```
gitignore  
  
local_s3/
```

12. Key Concepts Deep Dive

◆ Bag of Words (BoW)

- **Idea:** Represent text as word frequency vector
- **Steps:**
 1. Collect all unique words from corpus
 2. Take top N most frequent
 3. For each document, create vector of word counts
- **Limitation:** Ignores word order & context
- **Example:**

text

"I love this movie" → [1, 1, 1, 1, 0, 0, ...]

"I hate this movie" → [1, 1, 1, 1, 0, 0, ...] ← same vector!

◆ TF-IDF

- **TF (Term Frequency):** How often word appears in document
- **IDF (Inverse Document Frequency):** How rare word is across corpus
- **Formula:** $TF\text{-}IDF = TF \times \log(N / DF)$

- **Advantage:** Downweights common words ("the", "is"), upweights rare important words
- **Better than BoW** for many NLP tasks

◆ MLflow Nested Runs

text

Parent Run

```
|— Child Run 1 (LR + BoW)
|— Child Run 2 (LR + TF-IDF)
|— Child Run 3 (NB + BoW)
└— Child Run 4 (NB + TF-IDF)
```

- **Parent** = overall experiment
- **Child** = individual combination
- Enables **hierarchical comparison**

◆ Experiment Tracking Workflow

text

1. Define experiment (e.g., "LR Hyperparameter Tuning")
2. For each combination:
 - a. Start nested run
 - b. Log params (C, penalty, solver)
 - c. Train model
 - d. Log metrics (accuracy, f1)
 - e. End run
3. Compare all runs in MLflow UI
4. Pick best combination
5. Implement in production pipeline

◆ Why Not Always Pick "Best" Algorithm?

- Gradient Boosting performed best (accuracy-wise)
- But instructor chose **Logistic Regression** because:
 - Simpler to deploy
 - Faster inference

- Project structure built around it
- **Real-world lesson:** Best metric $\neq$ best choice for production

13. Common Pitfalls & Solutions

Problem	Solution
Cookiecutter folder nesting	Move files to root, delete empty folder
<code>src/models</code> conflicts with <code>models/</code>	Rename to <code>src/model/</code>
Credentials leaked in Git	Add <code>credentials.txt</code> to <code>.gitignore</code>
<code>FileNotFoundError</code> for <code>data.csv</code>	Use correct relative path or copy file
<code>ModuleNotFoundError</code> for local package	Run <code>pip install -e .</code>
Requirements conflict	Use instructor's <code>requirements.txt</code>
MLflow tracking URI outdated	Copy fresh URI from DagsHub
Notebook tracked with credentials	Add <code>notebooks/</code> to <code>.gitignore</code> temporarily

14. Git Workflow Summary

bash

Initial setup

```
git init
git add .
git commit -m "project structure ready"
git remote add origin <repo-url>
git push origin main
```

After experiments

```
git add .
git commit -m "experiments completed"
git push origin main
```

```
# Check status  
git status
```

Note: Notebooks with credentials are **not pushed** initially. Will be added after cleanup.

15. Final Takeaways

#	Takeaway
1	Cookiecutter standardizes project structure
2	DagsHub + MLflow = free remote experiment tracking
3	Never skip experiments — they inform production code
4	Balanced dataset prevents bias
5	Bag of Words vs TF-IDF — try both
6	Nested runs organize hyperparameter tuning
7	Best metric $\neq$ best production choice
8	Credentials must never be pushed to Git
9	DVC initializes data versioning early
10	Wait for EKS billing clarity before AWS deployment

16. What's Next?

Next Video: Build the complete ML pipeline & app in one shot

- Data ingestion
- Data preprocessing
- Model training
- Model evaluation
- Pipeline automation (DVC)

- Dockerization

Instructor's Note: Next video will be longer but contains most of the heavy lifting.

📌 End of Notes — Part 2



ओके सो वेलकम बैक टू दिस ...
TXT 156.12KB

make notes out of this transcript for me , explain all these topics properly with deep concept and code explanation

MLOps Capstone Project — Notes: Part 3 (Building the End-to-End ML Pipeline)

Scope of This Video

Build the **complete ML pipeline** from scratch:

1. **Logger module** (`src/logger.py`)
2. **Data ingestion** (S3 → local)
3. **Data preprocessing** (text cleaning)
4. **Feature engineering** (Bag of Words vectorization)
5. **Model building** (Logistic Regression training)
6. **Model evaluation** (metrics + MLflow tracking)
7. **Model registration** (MLflow Model Registry)
8. **DVC pipeline automation** (`dvc.yaml`)
9. **DVC remote push to S3** (data versioning)

Final outcome: One command (`dvc repro`) runs the entire pipeline and auto-registers models.

1. Project Architecture Recap

text

EXPERIMENTS (Previous Video)

|



EXPERIMENT TRACKING (MLflow + DagsHub)

|



FINALIZE STRATEGY ← Choose: Logistic Regression + Bag of Words

|



ML PIPELINE (THIS VIDEO)	
1. DATA INGESTION	→ data/raw/train.csv, test.csv
2. DATA PREPROCESSING	→ data/interim/train.csv
3. FEATURE ENGINEERING	→ data/processed/train_bow.csv + models/vectorizer.pkl
4. MODEL BUILDING	→ models/model.pkl
5. MODEL EVALUATION	→ reports/metrics.json
6. MODEL REGISTRATION	→ MLflow Model Registry
ALL ORCHESTRATED BY DVC (dvc.yaml)	

|



DVC PUSH → S3 REMOTE (Data Versioning)

2. Logger Module (src/logger.py)

Purpose

Centralized logging for debugging & production monitoring.

Code

```
python
```

```
import logging
import os
from datetime import datetime

# Create logs directory
LOG_DIR = "logs"
```

```
os.makedirs(LOG_DIR, exist_ok=True)

# Log file with timestamp
LOG_FILE = f"{datetime.now().strftime('%m_%d_%Y_%H_%M_%S')}.log"
LOG_FILE_PATH = os.path.join(LOG_DIR, LOG_FILE)

# Configure logging
logging.basicConfig(
    level=logging.DEBUG,
    format="[%(asctime)s] %(lineno)d %(name)s - %(levelname)s - %(message)s",
    handlers=[
        logging.FileHandler(LOG_FILE_PATH),
        logging.StreamHandler()
    ]
)

logger = logging.getLogger("mlops")
```

Key Concepts

Component	Purpose
FileHandler	Saves logs to file
StreamHandler	Prints logs to console
Formatter	Defines log message structure
Level DEBUG	Captures all levels (DEBUG, INFO, WARNING, ERROR)

3. Data Ingestion (src/data/data_ingestion.py)

Purpose

Fetch raw data from S3 → split → save to data/raw/

S3 Connection Class (src/connections/s3_connection.py)

python

```

import boto3
import pandas as pd
from src.logger import logger

class S3Operation:
    def __init__(self, bucket_name, access_key, secret_key, region="ap-south-1"):
        self.bucket_name = bucket_name
        self.s3_client = boto3.client(
            "s3",
            aws_access_key_id=access_key,
            aws_secret_access_key=secret_key,
            region_name=region
        )
        logger.info("Data ingestion from S3 bucket initialized")

    def fetch_file_from_s3(self, file_name):
        """Fetch a file from S3 and return as DataFrame."""
        obj = self.s3_client.get_object(Bucket=self.bucket_name, Key=file_name)
        df = pd.read_csv(obj["Body"])
        logger.info(f"Fetched {file_name} from S3 bucket with {len(df)} records")
        return df

```

Data Ingestion Pipeline

python

```

import os
import pandas as pd
from sklearn.model_selection import train_test_split
from src.logger import logger
from src.connections.s3_connection import S3Operation

def load_params(params_path):
    """Load params from params.yaml"""
    import yaml
    with open(params_path, "r") as f:
        params = yaml.safe_load(f)
    return params

def load_data(path):
    """Load CSV from a given path."""
    try:
        df = pd.read_csv(path)
        logger.info(f"Data loaded from {path}")
        return df

```

```

except Exception as e:
    logger.error(f"Error loading data: {e}")
    raise

def preprocess_data(df):
    """Keep only positive/negative, map to 0/1."""
    df = df[df["sentiment"].isin(["positive", "negative"])]
    df["sentiment"] = df["sentiment"].map({"positive": 1, "negative": 0})
    logger.info("Data preprocessing completed")
    return df

def save_data(train_df, test_df, data_path):
    """Save train/test splits."""
    os.makedirs(data_path, exist_ok=True)
    train_df.to_csv(os.path.join(data_path, "train.csv"), index=False)
    test_df.to_csv(os.path.join(data_path, "test.csv"), index=False)
    logger.info(f"Train and test data saved to {data_path}")

if __name__ == "__main__":
    # Load params
    params = load_params("params.yaml")
    test_size = params["data_ingestion"]["test_size"]

    # Fetch from S3 (or Local fallback)
    s3 = S3Operation(
        bucket_name="capstone-data-grid",
        access_key=os.getenv("AWS_ACCESS_KEY"),
        secret_key=os.getenv("AWS_SECRET_KEY")
    )
    df = s3.fetch_file_from_s3("data.csv")

    # Preprocess
    df = preprocess_data(df)

    # Split
    train_df, test_df = train_test_split(df, test_size=test_size, random_state=42)

    # Save
    save_data(train_df, test_df, "data/raw")

```

Key Concepts

Step

Why

Fetch from S3

Real-world data source

Step	Why
Filter labels	Remove neutral/unknown
Encode labels	ML models need numeric targets
Train-test split	Evaluate on unseen data
Save to raw/	DVC tracks this artifact

S3 Bucket Setup (AWS Console)

1. Go to **S3** → **Create Bucket**
2. Name: `capstone-data-grid` (must be globally unique)
3. Upload `data.csv`
4. Create **IAM User**:
 - IAM → Users → Create User → `capstone-user`
 - Attach policy: `AdministratorAccess` (for learning)
 - Security Credentials → Create Access Key → CLI
 - Save **Access Key** + **Secret Key**

4. Data Preprocessing (`src/data/data_preprocessing.py`)

Purpose

Clean text in `review` column → save to `data/interim/`

Text Cleaning Function

```
python
```

```
import re
import nltk
import pandas as pd
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from src.logger import logger
```

```
nlTK.download("stopwords", quiet=True)
nlTK.download("wordnet", quiet=True)

lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words("english"))

def preprocess_dataframe(df, column):
    """Apply text cleaning to a dataframe column."""
    df[column] = df[column].apply(clean_text)
    return df

def clean_text(text):
    """Clean text: lowercase, remove URLs, emails, punctuation, stopwords, lemmatize."""
    text = str(text).lower()
    text = re.sub(r"http\S+|www\S+|https\S+", "", text) # URLs
    text = re.sub(r"\S+@\S+", "", text) # Emails
    text = re.sub(r"[\w\s]", "", text) # Punctuation
    text = re.sub(r"\d+", "", text) # Numbers
    text = " ".join([w for w in text.split() if w not in stop_words])
    text = " ".join([lemmatizer.lemmatize(w) for w in text.split()])
    return text

if __name__ == "__main__":
    train_df = pd.read_csv("data/raw/train.csv")
    test_df = pd.read_csv("data/raw/test.csv")

    logger.info("Data loaded properly")

    train_df = preprocess_dataframe(train_df, "review")
    test_df = preprocess_dataframe(test_df, "review")

    os.makedirs("data/interim", exist_ok=True)
    train_df.to_csv("data/interim/train_processed.csv", index=False)
    test_df.to_csv("data/interim/test_processed.csv", index=False)

    logger.info("Data preprocessing completed and saved to data/interim")
```

Text Cleaning Steps Explained

Step	Example
Lowercase	"GREAT Movie!" → "great movie!"
Remove URLs	"visit http://x.com" → "visit "

Step	Example
Remove emails	"mail me@x.com" → "mail "
Remove punctuation	"great!" → "great"
Remove numbers	"movie 2024" → "movie "
Remove stopwords	"this is great" → "great"
Lemmatize	"running" → "run"

5. Feature Engineering (src/features/feature_engineering.py)

Purpose

Convert cleaned text → numeric vectors using **Bag of Words**

Code

python

```
import os
import pandas as pd
import pickle
from sklearn.feature_extraction.text import CountVectorizer
from src.logger import logger

def load_data(path):
    df = pd.read_csv(path)
    df.fillna("", inplace=True)
    return df

def apply_bag_of_words(X_train, X_test, max_features=50):
    """Fit CountVectorizer on train, transform both."""
    vectorizer = CountVectorizer(max_features=max_features)
    X_train_bow = vectorizer.fit_transform(X_train).toarray()
    X_test_bow = vectorizer.transform(X_test).toarray()
    return X_train_bow, X_test_bow, vectorizer

def save_data(train_df, test_df, data_path):
    os.makedirs(data_path, exist_ok=True)
```

```

train_df.to_csv(os.path.join(data_path, "train_bow.csv"), index=False)
test_df.to_csv(os.path.join(data_path, "test_bow.csv"), index=False)

if __name__ == "__main__":
    train_df = load_data("data/interim/train_processed.csv")
    test_df = load_data("data/interim/test_processed.csv")

    X_train = train_df["review"]
    y_train = train_df["sentiment"]
    X_test = test_df["review"]
    y_test = test_df["sentiment"]

    X_train_bow, X_test_bow, vectorizer = apply_bag_of_words(X_train, X_test, max_feature
s=50)

    # Build DataFrames with features + label
    train_bow_df = pd.DataFrame(X_train_bow)
    train_bow_df["label"] = y_train.values

    test_bow_df = pd.DataFrame(X_test_bow)
    test_bow_df["label"] = y_test.values

    # Save processed data
    save_data(train_bow_df, test_bow_df, "data/processed")

    # Save vectorizer for inference
    os.makedirs("models", exist_ok=True)
    with open("models/vectorizer.pkl", "wb") as f:
        pickle.dump(vectorizer, f)

    logger.info("Feature engineering completed. Vectorizer saved.")

```

⚠ Critical Concept: fit_transform VS transform

python

```

X_train_bow = vectorizer.fit_transform(X_train) # Learns vocab + transforms
X_test_bow   = vectorizer.transform(X_test)     # ONLY transforms (no Learning)

```

Why? If you `fit_transform` on test data, the vectorizer **learns test patterns** → **data leakage** → unrealistic performance.

Why Save the Vectorizer?

- During **inference**, user sends raw text
- Backend must transform text → same vector space
- Load `vectorizer.pkl` → `transform()` → feed to model

6. Model Building (`src/model/model_building.py`)

Purpose

Train Logistic Regression on Bag-of-Words features → save `model.pkl`

Code

```
python
```

```
import os
import pandas as pd
import pickle
from sklearn.linear_model import LogisticRegression
from src.logger import logger

def load_data(path):
    df = pd.read_csv(path)
    return df

def train_model(X_train, y_train):
    """Train Logistic Regression with best params from experiments."""
    classifier = LogisticRegression(
        C=10, penalty="l2", solver="liblinear"
    )
    classifier.fit(X_train, y_train)
    logger.info("Model training completed")
    return classifier

if __name__ == "__main__":
    train_df = load_data("data/processed/train_bow.csv")

    # Last column = label, rest = features
    X_train = train_df.iloc[:, :-1].values
    y_train = train_df.iloc[:, -1].values

    classifier = train_model(X_train, y_train)
```

```
os.makedirs("models", exist_ok=True)
with open("models/model.pkl", "wb") as f:
    pickle.dump(classifier, f)

logger.info("Model saved to models/model.pkl")
```

Best Params from Experiments

Parameter	Value
C	10
penalty	l2
solver	liblinear

This is the payoff of experimentation! Best hyperparameters found in Experiment 3 are now used in production.

7. Model Evaluation (src/model/model_evaluation.py)

Purpose

Evaluate model on test set → save metrics locally + log to MLflow

Code

python

```
import os
import json
import pandas as pd
import pickle
import mlflow
import mlflow.sklearn

from sklearn.metrics import accuracy_score, precision_score, recall_score, roc_auc_score
from src.logger import logger

# --- LOCAL MLflow Setup (for development) ---
os.environ["MLFLOW_TRACKING_URI"] = "https://dagshub.com/<user>/<repo>.mlflow"
```

```

os.environ["MLFLOW_TRACKING_USERNAME"] = "<username>"
os.environ["MLFLOW_TRACKING_PASSWORD"] = "<token>"

# --- PRODUCTION MLflow Setup (commented out) ---
# import dagshub
# dagshub.init(repo_owner="<user>", repo_name="<repo>", mlflow=True)

def load_model(path):
    with open(path, "rb") as f:
        model = pickle.load(f)
    return model

def load_data(path):
    df = pd.read_csv(path)
    X = df.iloc[:, :-1].values
    y = df.iloc[:, -1].values
    return X, y

def evaluate_model(model, X_test, y_test):
    """Compute metrics and return as dict."""
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1] # For AUC

    metrics = {
        "accuracy": accuracy_score(y_test, y_pred),
        "precision": precision_score(y_test, y_pred),
        "recall": recall_score(y_test, y_pred),
        "auc": roc_auc_score(y_test, y_prob)
    }
    return metrics

def save_metrics(metrics, path):
    os.makedirs(os.path.dirname(path), exist_ok=True)
    with open(path, "w") as f:
        json.dump(metrics, f, indent=4)
    logger.info(f"Metrics saved to {path}")

if __name__ == "__main__":
    model = load_model("models/model.pkl")
    X_test, y_test = load_data("data/processed/test_bow.csv")

    # Start MLflow experiment
    mlflow.set_experiment("my-dvc-pipeline")

    with mlflow.start_run():
        metrics = evaluate_model(model, X_test, y_test)
        save_metrics(metrics, "reports/metrics.json")

```

```
# Log metrics
for key, value in metrics.items():
    mlflow.log_metric(key, value)

# Log params
mlflow.log_param("model", "LogisticRegression")
mlflow.log_param("vectorizer", "BagOfWords")
mlflow.log_param("max_features", 50)
mlflow.log_param("test_size", 0.2)

# Log artifact
mlflow.log_artifact("reports/metrics.json")

# Log model
mlflow.sklearn.log_model(model, "model")

logger.info(f"Metrics logged to MLflow: {metrics}")
```

Metrics Explained

Metric	Formula	Meaning
Accuracy	$(TP+TN)/(TP+TN+FP+FN)$	Overall correctness
Precision	$TP/(TP+FP)$	Of predicted positives, how many are correct?
Recall	$TP/(TP+FN)$	Of actual positives, how many did we catch?
AUC	Area under ROC curve	Model's ability to distinguish classes

Why predict_proba for AUC?

- AUC needs **probabilities**, not hard labels
- `predict()` → 0 or 1
- `predict_proba()` → probability scores (e.g., 0.87)

8. Model Registration (src/model/register_model.py)

Purpose

Auto-register model to MLflow Model Registry → Stage = **Staging**

Code

python

```
import os
import json
import mlflow
from mlflow.tracking import MlflowClient
from src.logger import logger

# --- LOCAL MLflow Setup ---
os.environ["MLFLOW_TRACKING_URI"] = "https://dagshub.com/<user>/<repo>.mlflow"
os.environ["MLFLOW_TRACKING_USERNAME"] = "<username>"
os.environ["MLFLOW_TRACKING_PASSWORD"] = "<token>"

MODEL_NAME = "my_model"
MODEL_INFO_PATH = "reports/experiment_info.json"

def load_model_info(path):
    """Load run_id and model_path from JSON saved by evaluation step."""
    with open(path, "r") as f:
        model_info = json.load(f)
    return model_info

def register_model(model_name, model_info):
    """Register model to MLflow Model Registry and set stage to Staging."""
    model_uri = f"runs://{model_info['run_id']}/{model_info['model_path']}"
    model_version = mlflow.register_model(model_uri, model_name)

    client = MlflowClient()
    client.transition_model_version_stage(
        name=model_name,
        version=model_version.version,
        stage="Staging"
    )
    logger.info(f"Model '{model_name}' version {model_version.version} registered to Stag
ing")

if __name__ == "__main__":
    model_info = load_model_info(MODEL_INFO_PATH)
    register_model(MODEL_NAME, model_info)
```

MLflow Model Registry Stages

text

None → Staging → Production → Archived

Stage	Purpose
Staging	Newly trained, awaiting validation
Production	Live model serving real traffic
Archived	Deprecated models

Automation win: Previously, you clicked "Register Model" in MLflow UI manually. Now it's **code-driven**.

9. Saving experiment_info.json

In `model_evaluation.py` , after logging the model, save:

python

```
# Save run_id + model_path for registry step
experiment_info = {
    "run_id": mlflow.active_run().info.run_id,
    "model_path": "model"
}
with open("reports/experiment_info.json", "w") as f:
    json.dump(experiment_info, f)
```

This file is the **bridge** between evaluation and registration.

10. Params File (`params.yaml`)

Purpose

Centralize all hyperparameters → no hardcoding

Code

```
yaml

data_ingestion:
  test_size: 0.2

feature_engineering:
  max_features: 50
```

Loading in Code

```
python

import yaml

def load_params(path):
    with open(path, "r") as f:
        params = yaml.safe_load(f)
    return params

params = load_params("params.yaml")
test_size = params["data_ingestion"]["test_size"]
max_features = params["feature_engineering"]["max_features"]
```

11. DVC Pipeline (dvc.yaml)

Purpose

Define entire pipeline → one command runs everything

Code

```
yaml

stages:
  data_ingestion:
    cmd: python src/data/data_ingestion.py
    deps:
      - src/data/data_ingestion.py
      - src/connections/s3_connection.py
```

outs:

- data/raw/train.csv
- data/raw/test.csv

params:

- data_ingestion.test_size

data_preprocessing:

cmd: python src/data/data_preprocessing.py

deps:

- data/raw/train.csv
- data/raw/test.csv
- src/data/data_preprocessing.py

outs:

- data/interim/train_processed.csv
- data/interim/test_processed.csv

feature_engineering:

cmd: python src/features/feature_engineering.py

deps:

- data/interim/train_processed.csv
- data/interim/test_processed.csv
- src/features/feature_engineering.py

outs:

- data/processed/train_bow.csv
- data/processed/test_bow.csv
- models/vectorizer.pkl

params:

- feature_engineering.max_features

model_building:

cmd: python src/model/model_building.py

deps:

- data/processed/train_bow.csv
- src/model/model_building.py

outs:

- models/model.pkl

model_evaluation:

cmd: python src/model/model_evaluation.py

deps:

- models/model.pkl
- data/processed/test_bow.csv
- src/model/model_evaluation.py

outs:

- reports/metrics.json
- reports/experiment_info.json

```
register_model:
  cmd: python src/model/register_model.py
  deps:
    - reports/experiment_info.json
    - src/model/register_model.py
```

DVC Concepts

Key	Meaning
cmd	Command to run
deps	Inputs (files/params)
outs	Outputs (tracked artifacts)
params	Values from <code>params.yaml</code>

Run Pipeline

```
bash
```

```
dvc repro
```

DVC automatically:

1. Checks if deps changed (via MD5 hash)
2. Skips stages that are **up-to-date**
3. Re-runs only **changed stages + downstream**

This is automation + caching! Change `params.yaml` → DVC re-runs only affected stages.

12. DVC Lock File (`.dvc/lock`)

What It Stores

For each stage:

- **Code MD5** (script hash)
- **Deps MD5** (input hashes)
- **Outs MD5** (output hashes)

Why It Matters

```
bash
```

```
# Change a param
```

```
echo "test_size: 0.25" >> params.yaml
```

```
# DVC detects change
```

```
dvc status
```

```
# → data_ingestion changed
```

```
# Re-run only affected stages
```

```
dvc repro
```

13. DVC Remote Setup (S3)

Step 1: Install Dependencies

```
bash
```

```
pip install dvc[s3]
```

```
pip install awscli
```

Step 2: Remove Old Local Remote

```
bash
```

```
dvc remote list
```

```
# my_local /path/to/local_s3
```

```
dvc remote remove my_local
```

Step 3: Configure AWS CLI

```
bash
```

```
aws configure
# AWS Access Key ID: <your-access-key>
# AWS Secret Access Key: <your-secret-key>
# Default region name: ap-south-1
# Default output format: json
```

Step 4: Add S3 as DVC Remote

```
bash
```

```
dvc remote add -d my_remote s3://capstone-data-grid
```

Step 5: Push Data to S3

```
bash
```

```
dvc add data/          # or specific files
dvc commit
dvc push
```

Verification

- Go to **S3 Console** → capstone-data-grid bucket
- You'll see DVC-tracked artifacts (hashed folders)
- Original `data.csv` + DVC storage = complete data versioning

14. .gitignore Updates

```
gitignore
```

```
# Credentials
credentials.txt
```

```
# Data (tracked by DVC)
data/
local_s3/
```

```
# Models (tracked by DVC)
models/

# Logs
logs/

# Notebooks (contain credentials)
notebooks/
```

15. Key Concepts Deep Dive

◆ Why Save Vectorizer Separately?

- Model expects **numeric input**
- During inference: raw text → vectorizer → model
- If not saved → must re-fit vectorizer → **wrong vocabulary**

◆ Why reports/experiment_info.json ?

- MLflow `register_model` needs:
 - `run_id` (from evaluation run)
 - `model_path` (e.g., "model")
- JSON file = **contract** between eval & registry stages

◆ Why DVC + MLflow Together?

Tool	Responsibility
DVC	Data & pipeline versioning
MLflow	Experiment tracking & model registry
Together	Full reproducibility + model lifecycle

◆ Model Retraining Flow

text

1. New data arrives in S3
2. dvc repro → re-runs pipeline
3. New model trained → evaluated
4. MLflow registers new version (v2, v3...)
5. New version → Staging
6. (Later) Tests pass → Production

◆ dvc repro Output Example

text

```
Running stage 'data_ingestion':
> python src/data/data_ingestion.py
...
Running stage 'data_preprocessing':
> python src/data/data_preprocessing.py
...
Running stage 'model_building':
> python src/model/model_building.py
...
Running stage 'register_model':
> python src/model/register_model.py
Model 'my_model' version 3 registered to Staging
```

16. Common Errors & Fixes

Error	Fix
data.csv not found	Use correct relative path or copy to root
ModuleNotFoundError: src.connections	Ensure __init__.py exists in all folders
dvc repro says "data folder tracked by Git"	git rm -r --cached data/
MLflow credentials leaked	Use .gitignore + env vars
DVC remote permission denied	Check IAM user has S3 access
requirements.txt conflict	Use instructor's version

17. Final Project Flow

text

COMPLETE WORKFLOW	
1. EXPERIMENTS (Notebooks)	↳ MLflow tracking → pick best strategy
2. BUILD PIPELINE (src/)	↳ logger.py ↳ data_ingestion.py ← S3 ↳ data_preprocessing.py ← text cleaning ↳ feature_engineering.py ← Bag of Words ↳ model_building.py ← Logistic Regression ↳ model_evaluation.py ← metrics + MLflow ↳ register_model.py ← MLflow Registry
3. DVC PIPELINE (dvc.yaml)	↳ dvc repro → runs everything in order
4. DATA VERSIONING	↳ dvc add data/ ↳ dvc commit ↳ dvc push → S3
5. MODEL REGISTRY	↳ Auto-registered to Staging on every run
6. NEXT: CI/CD + Docker + EKS + Monitoring	

18. Final Takeaways

#	Takeaway
1	Logger module = centralized logging
2	S3 connection class = reusable data source

#	Takeaway
3	<code>fit_transform</code> on train , <code>transform</code> on test = no leakage
4	Save vectorizer for inference
5	Best params from experiments → production
6	<code>params.yaml</code> = no hardcoding
7	<code>dvc.yaml</code> = pipeline as code
8	<code>dvc repro</code> = one command, full pipeline
9	<code>experiment_info.json</code> = bridge between eval & registry
10	Auto-registration to Staging on every run
11	DVC + S3 = complete data versioning
12	Model retraining is now automated

19. Commands Cheat Sheet

bash

Environment

```
conda create -n atlas python=3.10 -y
conda activate atlas
```

Dependencies

```
pip install -r requirements.txt
pip install dvc[s3] awscli
```

DVC

```
dvc init
dvc repro          # Run full pipeline
dvc status         # Check what changed
dvc commit         # Commit outputs
dvc push           # Push to S3
dvc pull           # Pull from S3
```

AWS

```
aws configure
dvc remote add -d my_remote s3://capstone-data-grid

# Git
git add .
git commit -m "pipeline ready"
git push origin main

# Individual components (for debugging)
python src/data/data_ingestion.py
python src/data/data_preprocessing.py
python src/features/feature_engineering.py
python src/model/model_building.py
python src/model/model_evaluation.py
python src/model/register_model.py
```

20. What's Next?

Next Video: CI/CD pipeline + Docker + EKS deployment + Prometheus/Grafana monitoring

Instructor's Note: "You've now built a complete end-to-end ML pipeline with automation, model registry, and data versioning. This is the foundation. Next comes deployment — the part that puts you in the top 5%."

✦ End of Notes — Part 3



ओके सो हेलो एंड वेलकम बैक...
TXT 163.4KB

make notes out of this transcript for me , explain all these topics properly with deep concept and code explanation

MLOps Capstone Project — Notes: Part 4 (Flask App + CI/CD Pipeline)



Scope of This Video

1. **Build Flask app** (flask_app/) for frontend + prediction API
2. **Model serving** — dynamically fetch latest model from MLflow Registry
3. **Prometheus metrics** — expose request count, latency, prediction count
4. **CI/CD pipeline** — GitHub Actions workflow
5. **Model testing** — validate model performance before promotion
6. **Auto-promotion** — move best model from Staging → Production
7. **End-to-end CI/CD execution** — trigger, fail, debug, succeed

1. Progress Recap

text

- ✅ Project Introduction
- ✅ Experiments (tracked on DagsHub via MLflow API)
- ✅ End-to-end ML pipeline built
- ✅ Pipeline automated using DVC
- ✅ S3 configured as DVC remote (data versioning)
- 🔄 ← CURRENT: Flask App + CI/CD
- 🕒 Next: Docker + ECR + EKS + Prometheus/Grafana

2. Flask App Structure

text

```
flask_app/
├─ app.py                # Main Flask application
├─ templates/
│   └─ index.html        # Frontend HTML (LLM-generated)
├─ static/               # CSS/JS (if any)
├─ load_model_test.py    # Model loading test
├─ preprocessing_utility.py # Text cleaning utilities
└─ requirements.txt      # App-specific requirements (lighter than main)
```

⚠️ Why a Separate requirements.txt ?

- Main project requirements.txt is heavy (includes DVC, MLflow, sklearn, etc.)

- Docker image should be **minimal** — only what's needed to run the app
- Smaller image → faster builds, faster deployments, less storage

3. Flask App — app.py Deep Dive

3.1 Imports & Setup

```
python
```

```
from flask import Flask, request, render_template, jsonify
import mlflow
import mlflow.sklearn
import pickle
import pandas as pd
from prometheus_client import Counter, Histogram, generate_latest, CollectorRegistry, CON
TENT_TYPE_LATEST
import time
import os
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

nltk.download("stopwords", quiet=True)
nltk.download("wordnet", quiet=True)

app = Flask(__name__)
```

3.2 MLflow Connection (Two Modes)

Local Development Mode (Active):

```
python
```

```
os.environ["MLFLOW_TRACKING_URI"] = "https://dagshub.com/<user>/<repo>.mlflow"
os.environ["MLFLOW_TRACKING_USERNAME"] = "<username>"
os.environ["MLFLOW_TRACKING_PASSWORD"] = "<token>"
```

Production Mode (Commented — for CI/CD):

```
python
```

```
# import dagshub
# dagshub.init(repo_owner="<user>", repo_name="<repo>", mlflow=True)
```

Why two modes?

- Local: hardcoded credentials (for development)
- Production: environment variables (secure, CI/CD-friendly)

3.3 Text Preprocessing (Reused from Pipeline)

python

```
lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words("english"))

def normalize_text(text):
    text = str(text).lower()
    text = re.sub(r"http\S+|www\S+|https\S+", "", text)
    text = re.sub(r"\S+@\S+", "", text)
    text = re.sub(r"^\w\s]", "", text)
    text = re.sub(r"\d+", "", text)
    text = " ".join([w for w in text.split() if w not in stop_words])
    text = " ".join([lemmatizer.lemmatize(w) for w in text.split()])
    return text
```

Critical: Preprocessing must be **identical** to training. Any mismatch → wrong predictions.

3.4 Model Serving — Fetch Latest from Registry

python

```
def get_latest_model_version(model_name, stage="Staging"):
    """Fetch latest model version from MLflow Registry."""
    client = mlflow.tracking.MlflowClient()
    latest_version = client.get_latest_versions(model_name, stages=[stage])
    if not latest_version:
        raise ValueError(f"No model found in stage '{stage}'")
    return latest_version[0].version

MODEL_NAME = "my_model"
STAGE = "Production" # or "Staging" for testing
MODEL_VERSION = get_latest_model_version(MODEL_NAME, STAGE)
```

```

model_uri = f"models://{MODEL_NAME}/{MODEL_VERSION}"
model = mlflow.sklearn.load_model(model_uri)

# Load vectorizer (saved during feature engineering)
with open("models/vectorizer.pkl", "rb") as f:
    vectorizer = pickle.load(f)

print(f"Model version {MODEL_VERSION} loaded and serving")

```

Key Concept: Model Serving

text

Registry (Staging) → get_latest_versions() → model_uri → mlflow.load_model() → serve

Why this matters:

- Every retrain → new version in registry
- App always fetches **latest** version → no manual model swap
- Switch `STAGE` to "Production" → app serves best model only

3.5 Prometheus Metrics

python

```

registry = CollectorRegistry()

# Counter: total requests to app
REQUEST_COUNT = Counter(
    "app_request_count",
    "Total number of requests to the app",
    ["method", "endpoint", "http_status"],
    registry=registry
)

# Histogram: request latency
REQUEST_LATENCY = Histogram(
    "app_request_latency_seconds",
    "Request latency in seconds",
    ["endpoint"],
    registry=registry
)

```

```
# Counter: predictions per class
PREDICTION_COUNT = Counter(
    "app_prediction_count",
    "Count of predictions for each class",
    ["prediction_class"],
    registry=registry
)
```

 Metrics Explained

Metric	Type	Purpose
app_request_count	Counter	How many times endpoint hit?
app_request_latency_seconds	Histogram	Response time distribution
app_prediction_count	Counter	Positive vs Negative predictions

Why Prometheus?

- Scrapes /metrics endpoint periodically
- Grafana visualizes → dashboards, alerts
- Detects: traffic spikes, latency issues, model drift

3.6 Routes

Home Route:

```
python

@app.route("/")
def home():
    REQUEST_COUNT.labels(method="GET", endpoint="/", http_status=200).inc()
    return render_template("index.html")
```

Prediction Route:

```
python

@app.route("/predict", methods=["POST"])
def predict():
```

```

start_time = time.time()

# 1. Get input text
text = request.form.get("text", "")

# 2. Preprocess
cleaned_text = normalize_text(text)

# 3. Vectorize
features = vectorizer.transform([cleaned_text]).toarray()
feature_df = pd.DataFrame(features)

# 4. Predict
prediction = model.predict(feature_df)[0]
sentiment = "Positive" if prediction == 1 else "Negative"

# 5. Update metrics
PREDICTION_COUNT.labels(prediction_class=sentiment).inc()
REQUEST_LATENCY.labels(endpoint="/predict").observe(time.time() - start_time)
REQUEST_COUNT.labels(method="POST", endpoint="/predict", http_status=200).inc()

return render_template("index.html", prediction=sentiment)

```

Metrics Route (for Prometheus):

python

```

@app.route("/metrics")
def metrics():
    return generate_latest(registry), 200, {"Content-Type": CONTENT_TYPE_LATEST}

```

Prediction Flow

text

```

User Input → request.form → normalize_text() → vectorizer.transform()
           → DataFrame → model.predict() → sentiment → render_template
           → Prometheus metrics updated

```

3.7 Run the App

bash

```
python flask_app/app.py
# Open http://localhost:5000
```

Test:

- Input: "The movie was really really good" → Positive
- Input: "The movie was bad" → Negative

4. CI/CD Pipeline (GitHub Actions)

4.1 Workflow File Structure

text

```
.github/
├── workflows/
│   └── ci.yaml
```

4.2 CI Workflow (ci.yaml)

yaml

```
name: CI Pipeline

on:
  push:
    branches: [main]

jobs:
  project-testing:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Setup Python
        uses: actions/setup-python@v4
        with:
          python-version: "3.10"
```

- **name:** Cache pip dependencies
uses: actions/cache@v3
with:
 - path:** ~/.cache/pip
 - key:** \${{ runner.os }}-pip-\${{ hashFiles('requirements.txt') }}
- **name:** Install dependencies
run: |
 - python -m pip install --upgrade pip
 - pip install -r requirements.txt
- **name:** Run pipeline
env:
 - CAPSTONE_TEST:** \${{ secrets.CAPSTONE_TEST }}**run:** |
 - dvc repro
- **name:** Run model tests
env:
 - CAPSTONE_TEST:** \${{ secrets.CAPSTONE_TEST }}**run:** |
 - python -m pytest tests/test_model.py
- **name:** Promote model to production
env:
 - CAPSTONE_TEST:** \${{ secrets.CAPSTONE_TEST }}**run:** |
 - python scripts/promote_model.py
- **name:** Run Flask app tests
env:
 - CAPSTONE_TEST:** \${{ secrets.CAPSTONE_TEST }}**run:** |
 - python -m pytest tests/test_flask_app.py

4.3 Pipeline Stages Explained

Stage	Purpose
Checkout	Clone repo into GitHub runner
Setup Python	Install Python 3.10
Cache pip	Speed up subsequent runs

Stage	Purpose
Install deps	<code>pip install -r requirements.txt</code>
Run pipeline	<code>dvc repro</code> — runs full ML pipeline
Run model tests	Validate model performance thresholds
Promote model	Move Staging → Production
Run app tests	Validate Flask app works

4.4 GitHub Secrets Setup

text

Settings → Secrets and variables → Actions → New repository secret

Secret Name	Value
CAPSTONE_TEST	DagsHub token

How to generate DagsHub token:

- 1. DagsHub → Your Settings → Tokens
- 2. Generate New Token → Name: capstone-project
- 3. Copy token (shown only once!)
- 4. Paste into GitHub secret

5. Model Testing (tests/test_model.py)

Purpose

Validate model before promoting to Production.

Code

python

```

import os
import pickle
import pandas as pd
import mlflow
import pytest
from sklearn.metrics import accuracy_score, precision_score, recall_score

# MLflow connection
os.environ["MLFLOW_TRACKING_URI"] = "https://dagshub.com/<user>/<repo>.mlflow"
os.environ["MLFLOW_TRACKING_USERNAME"] = os.getenv("CAPSTONE_TEST")
os.environ["MLFLOW_TRACKING_PASSWORD"] = os.getenv("CAPSTONE_TEST")

MODEL_NAME = "my_model"
STAGE = "Staging"

def get_latest_model_version(model_name, stage):
    client = mlflow.tracking.MlflowClient()
    latest = client.get_latest_versions(model_name, stages=[stage])
    return latest[0].version if latest else None

def load_model(stage="Staging"):
    version = get_latest_model_version(MODEL_NAME, stage)
    model_uri = f"models://{MODEL_NAME}/{version}"
    model = mlflow.sklearn.load_model(model_uri)
    return model

def test_model_loaded():
    model = load_model(STAGE)
    assert model is not None, "Model failed to load"

def test_model_signature():
    """Test input/output shapes."""
    with open("models/vectorizer.pkl", "rb") as f:
        vectorizer = pickle.load(f)

    sample_text = "This movie was great"
    features = vectorizer.transform([sample_text]).toarray()

    # Input shape: (1, n_features)
    assert features.shape[0] == 1, "Input shape mismatch"
    assert features.shape[1] > 0, "No features generated"

    model = load_model(STAGE)
    prediction = model.predict(features)

    # Output shape: (1,)

```

```
assert len(prediction) == 1, "Output should be single prediction"
assert prediction[0] in [0, 1], "Prediction should be binary"

def test_model_performance():
    """Test model meets minimum performance thresholds."""
    model = load_model(STAGE)

    with open("models/vectorizer.pkl", "rb") as f:
        vectorizer = pickle.load(f)

    test_df = pd.read_csv("data/processed/test_bow.csv")
    X_test = test_df.iloc[:, :-1].values
    y_test = test_df.iloc[:, -1].values

    y_pred = model.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)

    # Thresholds (replace with production model's metrics)
    assert accuracy >= 0.60, f"Accuracy too low: {accuracy}"
    assert precision >= 0.60, f"Precision too low: {precision}"
    assert recall >= 0.60, f"Recall too low: {recall}"
```

Model Testing Concepts

Test	What It Validates
test_model_loaded	Model loads from registry
test_model_signature	Input/output shapes are correct
test_model_performance	Metrics above thresholds

Thresholds: Replace `0.60` with your production model's metrics. New model must beat these to be promoted.

6. Auto-Promotion Script (`scripts/promote_model.py`)

Purpose

Automatically move best model from **Staging** → **Production**.

Code

python

```
import os
import mlflow
from mlflow.tracking import MlflowClient

os.environ["MLFLOW_TRACKING_URI"] = "https://dagshub.com/<user>/<repo>.mlflow"
os.environ["MLFLOW_TRACKING_USERNAME"] = os.getenv("CAPSTONE_TEST")
os.environ["MLFLOW_TRACKING_PASSWORD"] = os.getenv("CAPSTONE_TEST")

MODEL_NAME = "my_model"
client = MlflowClient()

def get_latest_version(stage):
    versions = client.get_latest_versions(MODEL_NAME, stages=[stage])
    return versions[0] if versions else None

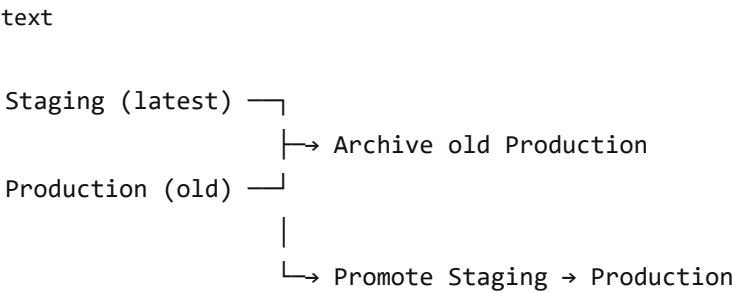
def promote_model():
    # 1. Get Latest Staging version
    staging_version = get_latest_version("Staging")
    if not staging_version:
        print("No model in Staging")
        return

    # 2. Archive current Production version (if any)
    production_version = get_latest_version("Production")
    if production_version:
        client.transition_model_version_stage(
            name=MODEL_NAME,
            version=production_version.version,
            stage="Archived"
        )
        print(f"Archived production version {production_version.version}")

    # 3. Promote Staging → Production
    client.transition_model_version_stage(
        name=MODEL_NAME,
        version=staging_version.version,
        stage="Production"
    )
    print(f"Model version {staging_version.version} promoted to Production")
```

```
if __name__ == "__main__":
    promote_model()
```

 Promotion Logic



MLflow Built-in Methods

Method	Purpose
get_latest_versions(name, stages)	Fetch latest version in stage
transition_model_version_stage()	Move model between stages

7. Flask App Tests (tests/test_flask_app.py)

Code

```
python

import pytest
from flask_app.app import app

@pytest.fixture
def client():
    app.config["TESTING"] = True
    with app.test_client() as client:
        yield client

def test_home_route(client):
    """Test home route returns 200."""
    response = client.get("/")
    assert response.status_code == 200
```

```

assert b"Sentiment Analysis" in response.data

def test_predict_route_positive(client):
    """Test prediction route with positive review."""
    response = client.post("/predict", data={"text": "I love this movie"})
    assert response.status_code == 200
    assert b"Positive" in response.data or b"Negative" in response.data

def test_predict_route_negative(client):
    """Test prediction route with negative review."""
    response = client.post("/predict", data={"text": "This movie was terrible"})
    assert response.status_code == 200
    assert b"Positive" in response.data or b"Negative" in response.data

def test_metrics_route(client):
    """Test Prometheus metrics endpoint."""
    response = client.get("/metrics")
    assert response.status_code == 200
    assert b"app_request_count" in response.data

```

Flask Testing Concepts

- `app.test_client()` — simulates requests without running server
- `client.get()` / `client.post()` — HTTP methods
- `assert b"..." in response.data` — bytes check

8. Debugging Journey (Real-World Lessons)

Error 1: `FileNotFoundError: data_ingestion.py`

Cause: `src/data/` was in `.gitignore` → not pushed to GitHub

Fix: Remove `src/` from `.gitignore`

Error 2: DVC tracking `data/` folder via Git

Cause: `data/` was tracked by both Git and DVC

Fix:

```
bash
```

```
git rm -r --cached data/
```

Add to `.gitignore` :

```
gitignore
```

```
data/
```

Error 3: Authentication Required (repeatedly)

Cause 1: Secret name mismatch (`CAPSTONE_TEST` VS `capstone_test`)

Cause 2: Token copied incorrectly

Cause 3: Code still using local MLflow setup instead of environment variables

Final Fix: Switch to production MLflow setup using env vars:

```
python
```

```
os.environ["MLFLOW_TRACKING_USERNAME"] = os.getenv("CAPSTONE_TEST")
```

```
os.environ["MLFLOW_TRACKING_PASSWORD"] = os.getenv("CAPSTONE_TEST")
```

Error 4: FAILED to reproduce data_ingestion

Cause: Output `data/raw/` already tracked by Git

Fix: `git rm -r --cached data/`

💡 Instructor's Debugging Wisdom

"When stuck, **zoom out**. Don't zoom in. The root cause is usually somewhere obvious you're overlooking."

"I went to the washroom, and the solution came to me. Sometimes stepping away is the best debugging strategy."

9. End-to-End CI/CD Success

Final Output

text

- ✓ Checkout code
- ✓ Setup Python
- ✓ Install dependencies
- ✓ Run pipeline (dvc repro)
- ✓ Run model tests
- ✓ Promote model to production
- ✓ Run Flask app tests

MLflow UI Result

- **Model Registry:** `my_model` versions 1–6
- **Version 6:** Now in **Production** stage
- **Version 5:** Archived automatically
- **Experiments:** New run added per pipeline execution

Flask App Serving

python

```
STAGE = "Production" # Changed from "Staging"
```

- App now serves **only** the Production model
- Prediction works end-to-end

10. Key Concepts Deep Dive

◆ Model Serving Patterns

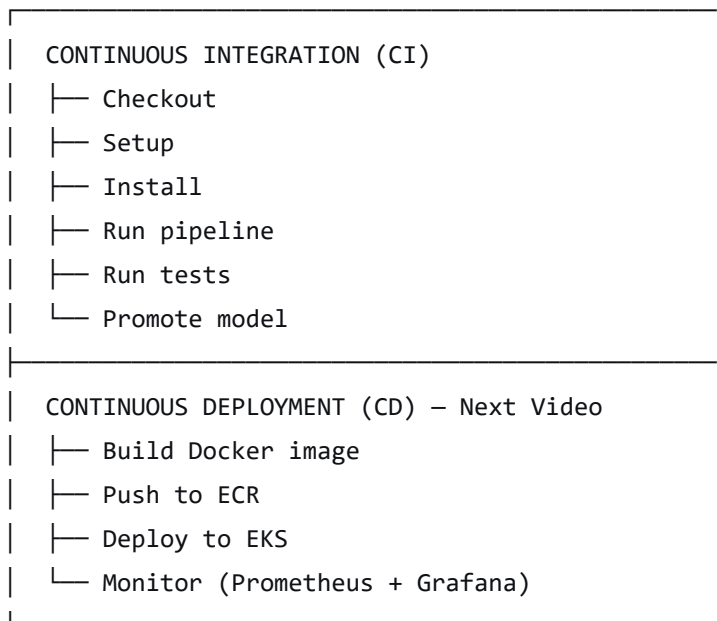
Pattern	Description
Static	Load model once at app startup
Dynamic	Fetch latest from registry on each request (or periodically)
This Project	Dynamic at startup — always serves latest production model

◆ Why Auto-Promotion?

- Manual promotion doesn't scale (50 runs/day = 50 manual clicks)
- Automation ensures: **best model always in production**
- Tests act as **gatekeepers** — bad models never reach users

◆ CI/CD Pipeline Stages

text



◆ Prometheus Metrics — Why These Three?

1. **Request Count** → Traffic volume, detect DDoS
2. **Latency Histogram** → Performance monitoring, SLO tracking
3. **Prediction Count** → Model behavior, drift detection

◆ DagsHub Token as Secret

- **Never** hardcode tokens in code
- GitHub Secrets → injected as env vars at runtime
- Rotate tokens periodically
- Delete leaked tokens immediately

11. Common Errors & Fixes

Error	Cause	Fix
FileNotFoundError: data_ingestion.py	src/ in .gitignore	Remove from .gitignore
data/raw/ tracked by Git	data/ tracked by both Git & DVC	git rm -r --cached data/
Authentication Required	Wrong secret name/token	Verify secret name + regenerate token
No model found in stage	Model not registered	Check MLflow Registry
Input shape mismatch	Vectorizer not applied	Apply vectorizer before predict
ModuleNotFoundError	Missing dependency	pip install -r requirements.txt

12. Final Project Status

text

PROJECT COMPLETION
✔ Experiments + MLflow tracking ✔ End-to-end ML pipeline ✔ DVC automation + S3 versioning ✔ Model Registry (auto-registration) ✔ Flask app + model serving ✔ Prometheus metrics exposed ✔ CI/CD pipeline (GitHub Actions) ✔ Model testing + auto-promotion 🕒 NEXT: Docker + ECR + EKS + Prometheus/Grafana

13. Commands Cheat Sheet

```
bash
```

```
# Run Flask app
```

```
python flask_app/app.py
```

```
# Run tests
```

```
pytest tests/test_model.py
```

```
pytest tests/test_flask_app.py
```

```
# Promote model manually
```

```
python scripts/promote_model.py
```

```
# Run pipeline
```

```
dvc repro
```

```
# Push to trigger CI/CD
```

```
git add .
```

```
git commit -m "ci: add tests"
```

```
git push origin main
```

```
# Check MLflow Registry
```

```
# → DagsHub UI → Models → my_model
```

14. Final Takeaways

#	Takeaway
1	Flask app serves model dynamically from Registry
2	Prometheus metrics expose app behavior
3	Separate requirements.txt for Docker efficiency
4	GitHub Secrets for secure token management
5	Model tests act as quality gates
6	Auto-promotion replaces manual model staging

#	Takeaway
7	<code>get_latest_versions()</code> fetches model by stage
8	<code>transition_model_version_stage()</code> moves models
9	Zoom out when debugging — root cause is often obvious
10	CI/CD fails many times before succeeding — that's normal
11	Env vars for production, hardcoded for local
12	End-to-end automation = true MLOps

15. What's Next?

Next Video: Docker + ECR + EKS Deployment

- Dockerize Flask app
- Push image to AWS ECR
- Deploy to EKS cluster
- Configure Prometheus + Grafana on EKS
- Monitoring & alerting setup

Instructor's Note: "You've built a fully functional 360° MLOps project. The next video adds cloud deployment — the final piece."

📌 **End of Notes — Part 4**



ओके सो हेलो एंड वेलकम बैक...
TXT 189.13KB

make notes out of this transcript for me , explain all these topics properly with deep concept and code explanation

MLOps Capstone Project — Notes: Part 5 (AWS EKS Deployment)

Scope of This Video

1. **Dockerize** the Flask app (build image locally)
2. **Push image to AWS ECR** (Elastic Container Registry)
3. **Set up AWS prerequisites** (IAM user, ECR repo, region)
4. **Configure GitHub Secrets** for CI/CD
5. **Extend CI/CD pipeline** to build & push Docker image
6. **EKS cluster creation** using `eksctl`
7. **Deploy Flask app to EKS** via `kubectl`
8. **Verify deployment** — access app via LoadBalancer URL
9. **AWS CloudFormation** explained (behind the scenes)

1. Progress Recap

text

- ✓ Project intro + architecture
- ✓ Experiments + MLflow tracking (DagsHub)
- ✓ End-to-end ML pipeline
- ✓ DVC automation + S3 data versioning
- ✓ CI/CD pipeline (up to Flask app tests)
- ✓ Model registry + auto-promotion
-  ← **CURRENT:** Docker + ECR + EKS Deployment
-  **Next:** Prometheus + Grafana monitoring on EKS

2. Docker Setup

2.1 Dockerfile

dockerfile

```
# Use slim Python image (smaller size)
FROM python:3.10-slim

# Set working directory
WORKDIR /app

# Copy Flask app contents
COPY flask_app/ /app/

# Copy vectorizer (needed for inference)
COPY models/vectorizer.pkl /app/models/vectorizer.pkl

# Install dependencies (optimized requirements)
RUN pip install --no-cache-dir -r requirements.txt

# Expose port
EXPOSE 5000

# PRODUCTION: Run with host binding (for Docker/EKS)
CMD ["python", "app.py"]

# LOCAL (commented out):
# CMD ["flask", "run", "--host=0.0.0.0", "--port=5000"]
```

2.2 Why Separate requirements.txt ?

Main requirements.txt	flask_app/requirements.txt
DVC, MLflow, sklearn, etc.	Flask, mlflow.sklearn, prometheus_client
Large image (~2GB+)	Small image (~500MB)
Not needed for serving	Only what app needs

Generating Optimized requirements.txt

```
bash

pip install pipreqs
cd flask_app
pipreqs . --force
```

```
pipreqs scans your imports → generates minimal requirements.txt
```

Instructor's advice: Use his provided file to avoid dependency conflicts.

3. Docker Build & Run (Local Test)

3.1 Build Image

```
bash
```

```
docker build -t flask-app .
```

Time: First build ~216 sec (downloading NLTK data), subsequent builds faster with cache.

3.2 Run Container

```
bash
```

```
docker run -p 5000:5000 \  
-e CAPSTONE_TEST=<your-dagshub-token> \  
flask-app
```

Expected Error: Authentication Required

Cause: App needs DagsHub token to fetch model from MLflow Registry.

Fix: Pass token as environment variable:

```
bash
```

```
docker run -p 5000:5000 -e CAPSTONE_TEST=<token> flask-app
```

Environment Variable Flow

```
text
```

```
GitHub Secret (CAPSTONE_TEST)  
→ CI/CD env var  
→ Docker run -e
```

```
→ os.getenv("CAPSTONE_TEST") in app.py
→ MLflow tracking auth
```

3.3 Verify Container

```
bash

# Check running containers
docker ps

# Test app
curl http://localhost:5000

# Stop container
docker stop <container_id>
```

3.4 Optional: Push to Docker Hub

```
bash

docker tag flask-app <username>/capstone-project:latest
docker push <username>/capstone-project:latest
```

Optional: Not required for this project (using ECR instead).

4. AWS Prerequisites

4.1 IAM User (Already Created)

Item	Value
Access Key	<AWS_ACCESS_KEY>
Secret Key	<AWS_SECRET_KEY>
Region	us-east-1 (or your default)
Permissions	AdministratorAccess + AmazonEC2ContainerRegistryFullAccess

Critical: ECR Permissions Fix

Problem: Admin access alone wasn't enough for ECR push.

Fix: Attach `AmazonEC2ContainerRegistryFullAccess` policy to IAM user:

text

IAM → Users → <user> → Permissions → Add permissions
→ Attach policies directly → `AmazonEC2ContainerRegistryFullAccess`

4.2 AWS Region

Check your default region:

bash

`aws configure`
Or check in AWS Console (top-right corner)

4.3 Create ECR Repository

text

AWS Console → Search "ECR" → Create repository

Setting	Value
Visibility	Private
Repository name	flask-app (or capstone-project)
Tag immutability	Disabled
Scan on push	Optional

Why ECR? AWS's Docker Hub alternative. Stores versioned images.

5. GitHub Secrets Setup

text

GitHub Repo → Settings → Secrets and variables → Actions → New repository secret

Secret Name	Value
CAPSTONE_TEST	DagsHub token
AWS_ACCESS_KEY_ID	IAM access key
AWS_SECRET_ACCESS_KEY	IAM secret key
AWS_DEFAULT_REGION	us-east-1
ECR_REPOSITORY	flask-app
AWS_ACCOUNT_ID	Your AWS account ID

 How to Get AWS Account ID

text

AWS Console → Top-right → Click username → Account ID

6. CI/CD Pipeline Extension

6.1 Extended ci.yaml

yaml

```
name: CI Pipeline

on:
  push:
    branches: [main]

jobs:
  project-testing:
    runs-on: ubuntu-latest
    steps:
```

```
# ... (previous steps: checkout, setup, install, run pipeline, tests, promote)
```

- **name:** Configure AWS credentials
uses: aws-actions/configure-aws-credentials@v2
with:
 - aws-access-key-id:** \${{ secrets.AWS_ACCESS_KEY_ID }}
 - aws-secret-access-key:** \${{ secrets.AWS_SECRET_ACCESS_KEY }}
 - aws-region:** \${{ secrets.AWS_DEFAULT_REGION }}

- **name:** Login to Amazon ECR
id: login-ecr
uses: aws-actions/amazon-ecr-login@v1

- **name:** Build, tag, and push Docker image to ECR
env:
 - ECR_REGISTRY:** \${{ steps.login-ecr.outputs.registry }}
 - ECR_REPOSITORY:** \${{ secrets.ECR_REPOSITORY }}
 - IMAGE_TAG:** latest**run:** |
 - docker build -t \$ECR_REGISTRY/\$ECR_REPOSITORY:\$IMAGE_TAG .
 - docker push \$ECR_REGISTRY/\$ECR_REPOSITORY:\$IMAGE_TAG

- **name:** Update kubeconfig
run: |
 - aws eks update-kubeconfig --region \${{ secrets.AWS_DEFAULT_REGION }} --name flask-app-cluster

- **name:** Create Kubernetes secret
run: |
 - kubectl create secret generic capstone-test \
 - from-literal=CAPSTONE_TEST=\${{ secrets.CAPSTONE_TEST }} \
 - dry-run=client -o yaml | kubectl apply -f -

- **name:** Deploy to EKS
run: |
 - kubectl apply -f deployment.yaml
 - kubectl apply -f service.yaml

6.2 Key Steps Explained

Step	Purpose
Configure AWS credentials	Authenticate GitHub runner to AWS
Login to ECR	Docker login to ECR registry

Step	Purpose
Build & push image	Create image, push to ECR
Update kubeconfig	Point kubectl to EKS cluster
Create K8s secret	Inject token as K8s secret
Deploy to EKS	Apply deployment + service YAMLs

7. Kubernetes Deployment Files

7.1 deployment.yaml

yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-app
  labels:
    app: flask-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: flask-app
  template:
    metadata:
      labels:
        app: flask-app
    spec:
      containers:
        - name: flask-app
          image: <AWS_ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/flask-app:latest
          ports:
            - containerPort: 5000
          env:
            - name: CAPSTONE_TEST
              valueFrom:
                secretKeyRef:
                  name: capstone-test
                  key: CAPSTONE_TEST
```

```
resources:
  requests:
    memory: "256Mi"
    cpu: "250m"
  limits:
    memory: "512Mi"
    cpu: "500m"
```

7.2 service.yaml

yaml

```
apiVersion: v1
kind: Service
metadata:
  name: flask-app-service
spec:
  selector:
    app: flask-app
  ports:
    - protocol: TCP
      port: 5000
      targetPort: 5000
  type: LoadBalancer
```

Key Concepts

Field	Meaning
replicas: 2	Run 2 pod instances (high availability)
containerPort: 5000	Container listens on port 5000
env from secretKeyRef	Inject secret as env var
resources	CPU/memory limits per pod
type: LoadBalancer	AWS provisions ELB → external URL

Why LoadBalancer?

- Pods have **internal IPs** (not accessible externally)

- LoadBalancer service provisions **AWS ELB**
- ELB gives a **public DNS** → accessible from internet
- Traffic → ELB → routes to any healthy pod

8. AWS EKS Cluster Setup

8.1 Prerequisites (System Setup)

Tool	Purpose	Check
AWS CLI	Talk to AWS	<code>aws --version</code>
kubect1	Talk to K8s cluster	<code>kubect1 version --client</code>
eksctl	Create EKS cluster	<code>eksctl version</code>

8.2 Installing eksctl (Windows)

bash

Download eksctl

```
curl -LO "https://github.com/eksctl-io/eksctl/releases/latest/download/eksctl_Windows_amd64.zip"
```

Extract

```
tar -xzf eksctl_Windows_amd64.zip
```

Move to system path (e.g., C:\Windows\System32)

```
move eksctl.exe C:\Windows\System32\
```

8.3 Installing kubect1 (Windows)

bash

```
curl -LO "https://dl.k8s.io/release/v1.28.0/bin/windows/amd64/kubect1.exe"
```

```
move kubect1.exe C:\Windows\System32\
```



Common Conflict: AWS CLI in Anaconda

Problem: `aws` command points to Anaconda's Python-based AWS CLI (old version).

Fix:

```
bash
```

```
pip uninstall awscli
# Install AWS CLI v2 from official installer
```

Add to System PATH:

```
text
```

```
Control Panel → System → Advanced System Settings
→ Environment Variables → System Variables → Path → Add:
C:\Program Files\Amazon\AWSCLIV2\
```

8.4 Create EKS Cluster

```
bash
```

```
eksctl create cluster \
  --name flask-app-cluster \
  --region us-east-1 \
  --nodegroup-name flask-app-nodes \
  --node-type t3.small \
  --nodes 1 \
  --managed
```

Flags Explained

Flag	Purpose
<code>--name</code>	Cluster name
<code>--region</code>	AWS region
<code>--nodegroup-name</code>	Worker node group name
<code>--node-type</code>	EC2 instance type (<code>t3.small</code> — cheap)
<code>--nodes</code>	Number of worker nodes

Flag	Purpose
--managed	AWS manages health checks, updates

Time: 15–30 minutes (first time)

8.5 Verify Cluster

```
bash
```

```
# List clusters
```

```
aws eks list-clusters
```

```
# Check cluster status
```

```
aws eks describe-cluster --name flask-app-cluster --query "cluster.status"
```

```
# Update kubeconfig
```

```
aws eks update-kubeconfig --region us-east-1 --name flask-app-cluster
```

```
# Check nodes
```

```
kubectl get nodes
```

```
# → Should show 1 node in Ready state
```

```
# Check namespaces
```

```
kubectl get namespaces
```

```
# → default, kube-system, kube-public, kube-node-lease
```

```
# Check pods
```

```
kubectl get pods
```

```
# → No resources (nothing deployed yet)
```

```
# Check services
```

```
kubectl get svc
```

```
# → Only kubernetes service (default)
```

9. Security Group Configuration

Purpose

Allow external traffic to reach the app on port **5000**.

Steps

text

AWS Console → EC2 → Instances → Click on EKS worker node
→ Security tab → Click Security Group
→ Edit inbound rules → Add rule:
 Type: Custom TCP
 Port range: 5000
 Source: Anywhere (0.0.0.0/0)
→ Save rules

Why This Works for Scaling

- Security group is attached to the **node group**
- Adding more nodes → same security group applied
- **No need** to reconfigure per node

10. AWS CloudFormation (Behind the Scenes)

What is CloudFormation?

"AWS CloudFormation is a service that helps define and provision AWS infrastructure as code using templates."

How It Relates to EKS

text

eksctl create cluster
 ↓ (behind the scenes)
CloudFormation Stack 1: eksctl-flask-app-cluster-cluster
 → Creates EKS control plane
CloudFormation Stack 2: eksctl-flask-app-cluster-nodegroup-flask-app-nodes
 → Creates worker nodes (EC2 instances)
 → Creates VPC, subnets, security groups, IAM roles

Key Concept: Stack

- **Stack** = logical grouping of AWS resources
- Can contain: VPC, subnets, EC2 instances, IAM roles, etc.
- Managed as a single unit

Why This Matters

- You didn't manually create VPC/subnets/EC2
- `eksctl` generated CloudFormation templates
- CloudFormation provisioned everything
- **If a node dies**, CloudFormation auto-recreates it

Verify in Console

text

AWS Console → CloudFormation → Stacks

→ See: `eksctl-flask-app-cluster-cluster`

→ See: `eksctl-flask-app-cluster-nodegroup-flask-app-nodes`

text

AWS Console → EC2 → Instances

→ See: `t3.small` instance named "flask-app-nodes"

11. Deployment Verification

11.1 Check Pods

bash

`kubectl get pods`

#	NAME	READY	STATUS	RESTARTS	AGE
#	<code>flask-app-7d8f9c6b5-abc12</code>	<code>1/1</code>	<code>Running</code>	<code>0</code>	<code>2m</code>
#	<code>flask-app-7d8f9c6b5-def34</code>	<code>1/1</code>	<code>Running</code>	<code>0</code>	<code>2m</code>

11.2 Check Services

```
bash
```

```
kubectl get svc
```

# NAME	TYPE	CLUSTER-IP	EXTERNAL-IP
# flask-app-service	LoadBalancer	10.100.10.100	a1b2c3.elb.amazonaws.com
# kubernetes	ClusterIP	10.100.0.1	<none>

11.3 Get External URL

```
bash
```

```
kubectl get svc flask-app-service -o jsonpath='{.status.loadBalancer.ingress[0].hostname}'  
# → a1b2c3-1234567890.us-east-1.elb.amazonaws.com
```

11.4 Access App

```
text
```

```
http://<EXTERNAL-IP>:5000
```

Test with:

- "The movie was really good" → Positive/Negative
- "Movie was a must watch for kids" → Prediction returned

Note: Model is basic (accuracy ~60%) — predictions may not be accurate. Focus is on deployment working.

11.5 Verify with curl

```
bash
```

```
curl http://<EXTERNAL-IP>:5000  
# → Returns HTML (200 OK)
```

12. Debugging Journey (Real-World Lessons)

Error 1: ImagePullBackOff

Cause: ECR repository name mismatch (flask-app VS capstone-app)

Fix: Renamed ECR repo to match deployment YAML

Error 2: Authentication Required (Docker run)

Cause: Missing CAPSTONE_TEST env var

Fix:

```
bash
```

```
docker run -p 5000:5000 -e CAPSTONE_TEST=<token> flask-app
```

Error 3: Authentication Required (CI/CD)

Cause 1: Secret name typo (CAPSTONE_TEST VS capstone_test)

Cause 2: Token copied incorrectly

Cause 3: Code still using local MLflow setup

Fix: Switch to production MLflow setup:

```
python
```

```
os.environ["MLFLOW_TRACKING_USERNAME"] = os.getenv("CAPSTONE_TEST")  
os.environ["MLFLOW_TRACKING_PASSWORD"] = os.getenv("CAPSTONE_TEST")
```

Error 4: .gitignore blocked src/ folder

Cause: src/data/ was in .gitignore

Fix: Remove from .gitignore , commit, push

Error 5: DVC tracking data/ via Git

Fix:

```
bash
```

```
git rm -r --cached data/
```

💡

Instructor's Debugging Wisdom

"95% of people watching this series will give up. You just need to make sure you're in that 5% category."

"When stuck, **zoom out**. Don't zoom in. The root cause is usually obvious."

"You WILL face difficulties. I can guarantee this. But if you complete this end-to-end, you're in the top 5%."

13. Key Concepts Deep Dive

◆ Docker Image Lifecycle

text

Dockerfile → docker build → Image → docker run → Container
→ docker push → Registry (ECR/DockerHub)

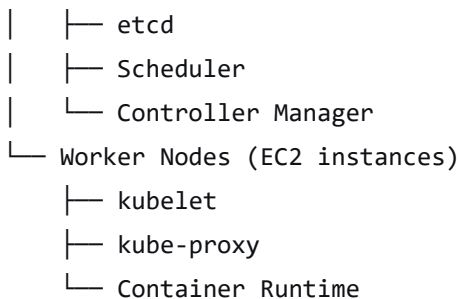
◆ ECR vs Docker Hub

Feature	ECR	Docker Hub
Provider	AWS	Docker
Auth	IAM	Docker account
Integration	Native AWS	Manual
Private repos	Free (500MB)	Paid
Best for	AWS deployments	Public images

◆ EKS Architecture

text

EKS Cluster
├─ Control Plane (AWS managed)
|
└─ API Server



◆ **Kubernetes Objects Used**

Object	Purpose
Deployment	Manages pod replicas, rolling updates
Service (LoadBalancer)	Exposes pods via AWS ELB
Secret	Stores sensitive data (tokens)
Pod	Smallest deployable unit

◆ **Why 2 Replicas?**

- **High availability** — if one pod dies, traffic continues
- **Load balancing** — ELB distributes traffic
- **Zero-downtime deploys** — rolling updates

◆ **Resource Limits**

```
yaml
resources:
  requests:
    memory: "256Mi"
    cpu: "250m"
  limits:
    memory: "512Mi"
    cpu: "500m"
```

- **Requests:** Minimum guaranteed resources
- **Limits:** Maximum allowed

- 250m = 250 millicores = 0.25 CPU

14. Common Errors & Fixes

Error	Cause	Fix
ImagePullBackOff	Wrong image name/tag	Check ECR repo name in YAML
Authentication Required	Missing token	Add CAPSTONE_TEST env var
CrashLoopBackOff	App crashing on start	Check kubectl logs <pod>
Pending pod	Insufficient resources	Increase node size or reduce requests
ErrImagePull	ECR permission denied	Add ECR policy to IAM user
aws: command not found	AWS CLI not in PATH	Add to system PATH
eksctl: command not found	eksctl not in PATH	Move to C:\windows\System32\
kubectl: connection refused	kubeconfig not set	aws eks update-kubeconfig

15. Final Project Status

text

PROJECT COMPLETION
✓ Experiments + MLflow tracking ✓ End-to-end ML pipeline ✓ DVC automation + S3 versioning ✓ Model Registry (auto-registration) ✓ Flask app + model serving ✓ Prometheus metrics exposed ✓ CI/CD pipeline (GitHub Actions) ✓ Model testing + auto-promotion ✓ Docker image build + push to ECR

- ✓ EKS cluster + deployment
- ✓ App accessible via LoadBalancer URL
- ⌚ NEXT: Prometheus + Grafana monitoring on EKS

16. Commands Cheat Sheet

bash

Docker

```
docker build -t flask-app .
```

```
docker run -p 5000:5000 -e CAPSTONE_TEST=<token> flask-app
```

```
docker ps
```

```
docker stop <container_id>
```

AWS

```
aws configure
```

```
aws eks list-clusters
```

```
aws eks update-kubeconfig --region us-east-1 --name flask-app-cluster
```

ECR

```
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-st  
din <account-id>.dkr.ecr.us-east-1.amazonaws.com
```

EKS

```
eksctl create cluster --name flask-app-cluster --region us-east-1 --nodegroup-name flask-  
app-nodes --node-type t3.small --nodes 1 --managed
```

Kubernetes

```
kubectl get nodes
```

```
kubectl get pods
```

```
kubectl get svc
```

```
kubectl apply -f deployment.yaml
```

```
kubectl apply -f service.yaml
```

```
kubectl logs <pod-name>
```

```
kubectl describe pod <pod-name>
```

17. Final Takeaways

#	Takeaway
1	Docker packages app + dependencies
2	Optimized requirements = smaller image
3	ECR = AWS's Docker registry
4	IAM permissions must include ECR access
5	GitHub Secrets for all AWS credentials
6	CI/CD extends to Docker build + ECR push
7	eksctl creates cluster via CloudFormation
8	CloudFormation manages AWS infra as code
9	Security group must open port 5000
10	LoadBalancer service gives public URL
11	2 replicas = high availability
12	Debugging takes time — zoom out, don't give up
13	<code>ImagePullBackOff</code> = wrong image name
14	Secrets in K8s inject env vars securely
15	End-to-end works — app accessible via URL

18. What's Next?

Next Video: Prometheus + Grafana on EKS

- Deploy Prometheus to EKS cluster
- Deploy Grafana to EKS cluster
- Connect to Flask app's `/metrics` endpoint
- Build dashboards for monitoring

- Set up alerts

Instructor's Note: "You've built a fully deployed, production-grade MLOps project on AWS EKS. Next comes monitoring — the final piece that makes it truly production-ready."

📌 End of Notes — Part 5



ओके अ सो वेलकम बैक टू यू ...
TXT 89KB

make notes out of this transcript for me , explain all these topics properly with deep concept and code explanation

📘 MLOps Capstone Project — Notes: Part 6 (Prometheus + Grafana Monitoring + AWS Cleanup)

🎯 Scope of This Video

1. Set up Prometheus server on AWS EC2
2. Set up Grafana server on AWS EC2
3. Connect Prometheus → Flask app (scrape /metrics)
4. Connect Grafana → Prometheus (data source)
5. Build Grafana dashboards (request count, predictions)
6. AWS cleanup — delete all resources safely
7. CloudFormation deep dive
8. Fleet Request quota issue explained
9. PVC error — why Prometheus/Grafana failed inside EKS

1. Progress Recap

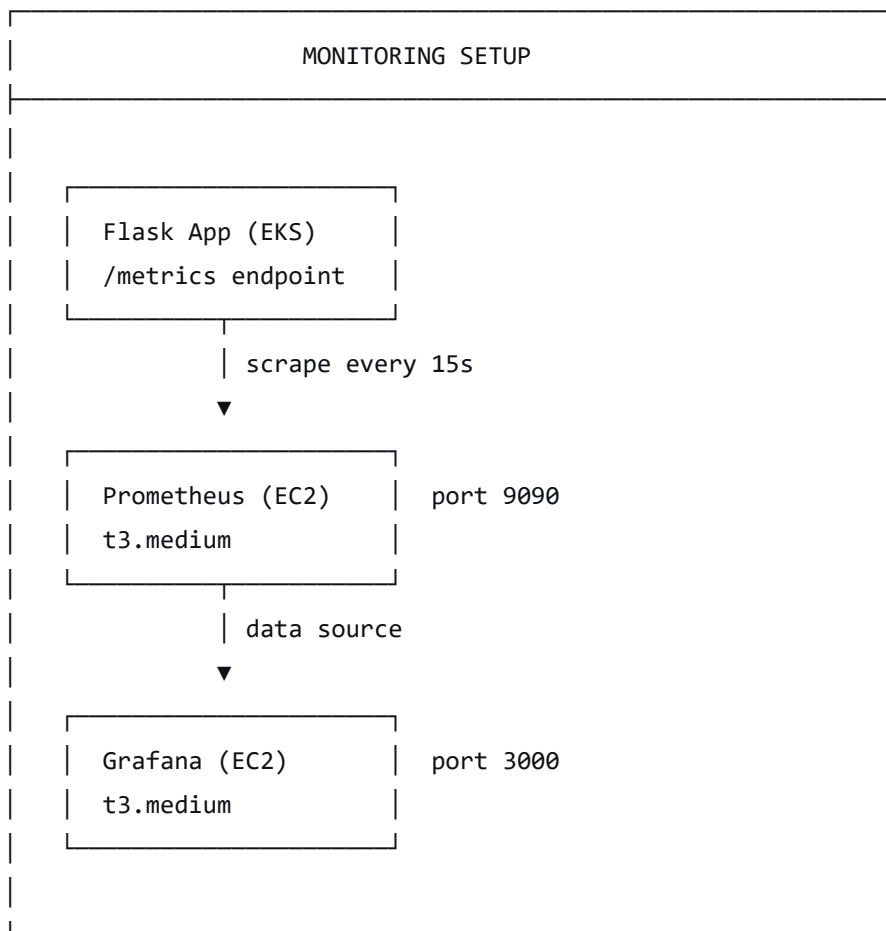
text

- ✅ Experiments + MLflow tracking (DagsHub)
- ✅ End-to-end ML pipeline (DVC)

- ✓ CI/CD pipeline (GitHub Actions)
- ✓ Model Registry + auto-promotion
- ✓ Docker + ECR
- ✓ EKS deployment (Flask app)
- 🔗 ← CURRENT: Prometheus + Grafana monitoring
- 🔗 ← CURRENT: AWS cleanup

2. Monitoring Architecture

text



Why Separate EC2 Instances?

- Prometheus/Grafana deployment **inside EKS failed** due to PVC errors
- Solution: Deploy on **separate EC2 instances**
- Common industry practice when PVCs aren't available

3. EC2 Instance Setup (Prometheus)

3.1 Launch Instance

text

AWS Console → EC2 → Launch Instance

Setting	Value
Name	prometheus-server
AMI	Ubuntu 20.04
Instance type	t3.medium
Storage	20 GB SSD
Security group	Allow SSH (22), Prometheus (9090)

Cost Estimate

- t3.medium = **\$0.06/hour** (~₹5/hour)
- 2 hours of testing = ~₹10–20
- **Delete immediately after use**

3.2 Connect via SSH

bash

```
# Via EC2 Instance Connect (browser)
# Or via SSH
ssh -i <key.pem> ubuntu@<public-ip>
```

3.3 Install Prometheus

bash

```
# 1. Update packages
sudo apt-get update && sudo apt-get upgrade -y
```

2. Download Prometheus (check latest version)

```
wget https://github.com/prometheus/prometheus/releases/download/v2.45.0/prometheus-2.45.0.linux-amd64.tar.gz
```

3. Extract

```
tar -xvf prometheus-2.45.0.linux-amd64.tar.gz
```

4. Move to standard location

```
sudo mv prometheus-2.45.0.linux-amd64 /usr/local/prometheus
```

```
sudo mv /usr/local/prometheus/prometheus /usr/local/bin/
```

```
sudo mv /usr/local/prometheus/promtool /usr/local/bin/
```

5. Verify

```
prometheus --version
```

3.4 Configure Prometheus (prometheus.yml)

yaml

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]

  - job_name: "flask-app"
    static_configs:
      - targets: ["<EKS-LOADBALANCER-URL>:5000"]
```

Key Config Explained

Field	Meaning
scrape_interval: 15s	Prometheus scrapes every 15 seconds
job_name	Identifier for target
targets	URL to scrape (/metrics auto-appended)

3.5 Run Prometheus

bash

Find binary location

`which prometheus`

→ /usr/local/bin/prometheus

Run with config

`prometheus --config.file=/usr/local/prometheus/prometheus.yml`

3.6 Access Prometheus

text

`http://<EC2-PUBLIC-IP>:9090`

Verify:

- Go to **Status** → **Targets**
- `flask-app` target should be **UP**
- Go to **Graph** → query `app_request_count_total`
- See metrics scraped from EKS app

4. EC2 Instance Setup (Grafana)

4.1 Launch Instance

Same as Prometheus:

- Name: `grafana-server`
- Instance type: `t3.medium`
- Storage: 10 GB
- Security group: Allow SSH (22), Grafana (3000)

4.2 Install Grafana

```
bash

# 1. Update packages
sudo apt-get update && sudo apt-get upgrade -y

# 2. Install dependencies
sudo apt-get install -y apt-transport-https software-properties-common wget

# 3. Add Grafana GPG key
sudo wget -q -O /usr/share/keyrings/grafana.key https://apt.grafana.com/gpg.key

# 4. Add Grafana repo
echo "deb [signed-by=/usr/share/keyrings/grafana.key] https://apt.grafana.com stable main" | sudo tee -a /etc/apt/sources.list.d/grafana.list

# 5. Update and install
sudo apt-get update
sudo apt-get install -y grafana

# 6. Start & enable service
sudo systemctl start grafana-server
sudo systemctl enable grafana-server

# 7. Check status
sudo systemctl status grafana-server
```

 Key Concepts

Command	Purpose
systemctl start	Start service now
systemctl enable	Auto-start on boot
systemctl status	Check if running

4.3 Access Grafana

```
text

http://<EC2-PUBLIC-IP>:3000
```

Default credentials:

- Username: admin
- Password: admin
- → Prompted to change (can skip for testing)

5. Connect Grafana → Prometheus

Steps

text

Grafana → Connections → Data Sources → Add Data Source
→ Select "Prometheus"
→ URL: `http://<PROMETHEUS-IP>:9090`
→ HTTP Method: POST
→ Save & Test

Common Error: "Data source is not working"

Cause: Wrong URL format

Fix: Use `http://<IP>:9090` — no trailing slash, no extra characters.

6. Build Grafana Dashboards

6.1 Dashboard 1: Request Count

text

Grafana → Dashboards → New Dashboard → Add Visualization
→ Select Prometheus data source
→ Query: `app_request_count_total`
→ Panel title: "Request to Home Route"

6.2 Query Customization

Setting	Value
Metric	app_request_count_total
Label filter	endpoint="/" , method="GET"
Aggregation	rate (per second)
Time range	Last 5 minutes
Legend	{{method}} {{endpoint}}

6.3 Dashboard 2: Prediction Count

text

Query: app_prediction_count_total
Label filter: prediction_class="Positive" or "Negative"

 PromQL Examples

```
promql

# Total requests to home route
app_request_count_total{endpoint="/", method="GET"}

# Request rate per second
rate(app_request_count_total[5m])

# Predictions by class
app_prediction_count_total{prediction_class="Positive"}
```

7. AWS Cleanup (CRITICAL)

7.1 Stop & Terminate EC2 Instances

text

EC2 → Instances → Select prometheus-server & grafana-server

→ Instance State → Stop → Terminate

7.2 Delete EKS Cluster

bash

```
eksctl delete cluster --name flask-app-cluster --region us-east-1
```

Time: 10–15 minutes

What it deletes:

- EKS control plane
- Worker nodes (EC2)
- VPC, subnets, security groups
- CloudFormation stacks

7.3 Delete ECR Repository

text

ECR → Repositories → Select repo → Delete

7.4 Delete S3 Bucket (Optional)

text

S3 → Select bucket → Empty → Delete

7.5 Verify Cleanup

bash

```
# Check clusters
```

```
aws eks list-clusters --region us-east-1
```

```
# → Should return empty
```

```
# Check EC2 instances
```

```
aws ec2 describe-instances --filters "Name=instance-state-name,Values=running" --query "R
```

```
ervations[].Instances[].InstanceId"  
# → Should return empty
```

7.6 Contact AWS Support

"After deleting everything, initiate a chat with AWS Support to confirm all chargeable services are terminated."

Message template:

text

I've stopped using my AWS account and deleted all resources.
Can you confirm that all chargeable services are terminated?

Response:

- They confirm no chargeable services are active
- Charges for used time will reflect in 24 hours
- **One-time waiver** if you explain it was for learning

8. Deep Dive: CloudFormation

What is CloudFormation?

"AWS CloudFormation is a service that helps define and provision AWS infrastructure as code using templates."

How EKS Uses It

text

```
eksctl create cluster
```

↓

CloudFormation Stack 1: eksctl-flask-app-cluster-cluster

→ EKS control plane

CloudFormation Stack 2: eksctl-flask-app-cluster-nodegroup-flask-app-nodes

→ Worker nodes (EC2)

→ VPC, subnets, security groups, IAM roles

Key Concepts

Term	Meaning
Stack	Logical grouping of AWS resources
Template	JSON/YAML file defining resources
Provisioning	Creating resources from template

Verify in Console

text

AWS Console → CloudFormation → Stacks

→ See: eksctl-flask-app-cluster-cluster

→ See: eksctl-flask-app-cluster-nodegroup-flask-app-nodes

9. Deep Dive: Fleet Request Quota Issue

What is a Fleet Request?

"A Fleet Request in AWS is an internal process for provisioning EC2 instances."

How It Works

text

eksctl create cluster

↓

EKS uses Auto Scaling Group (ASG)

↓

ASG sends Fleet Request to EC2

↓

EC2 provisions instances

Error: "You have reached your quota for maximum Fleet Requests"

Cause: AWS account quota limits on concurrent fleet requests

Fix:

1. Contact AWS Support via chat
2. Request quota increase
3. Explain it's for learning/development
4. They resolve billing account issues



Instructor's Experience

"I spent 1-2 weeks debugging this. Finally contacted AWS Support and they fixed my billing account."

10. Deep Dive: PVC Error (Why Prometheus Failed Inside EKS)

What is PVC?

"PVC (Persistent Volume Claim) is a request for storage by a Kubernetes application."

How It Works

text

```
Pod (Prometheus/Grafana)
  ↓ requests storage
PVC (Persistent Volume Claim)
  ↓ bound to
PV (Persistent Volume)
  ↓ provisioned by
Storage Class (EBS, NFS, etc.)
```



The Error

- Prometheus/Grafana deployed inside EKS failed
- PVC request was **denied**
- Storage class couldn't provision EBS volume

Why It Failed

- EKS default storage class may not be configured
- EBS CSI driver may not be installed
- Permissions issue with storage provisioning

Solution

- Deploy Prometheus/Grafana on **separate EC2 instances**
- This is also **common in production** when PVCs aren't available

Key Takeaway

"If PVC errors occur, deploy monitoring tools on separate EC2 instances. This is common industry practice."

11. Key Concepts Deep Dive

◆ Prometheus Pull Model

text

Prometheus → HTTP GET /metrics → Target
↑
└─ Every 15 seconds

Why pull?

- Prometheus controls scrape frequency
- Targets just expose metrics
- No push logic needed in app

◆ Prometheus Metrics Types

Type	Use Case
Counter	Total requests, predictions

Type	Use Case
Gauge	Current memory, CPU
Histogram	Request latency distribution
Summary	Quantiles (p50, p95, p99)

◆ Grafana Dashboard Best Practices

- One dashboard per app
- Group related panels
- Use variables for filtering
- Set time ranges (last 5m, 1h, 24h)
- Add alerts based on thresholds

◆ AWS Billing Safety

- Set billing alarms (AWS Budgets)
- Delete resources immediately after use
- Contact support to confirm cleanup
- Don't leave clusters running overnight

12. Commands Cheat Sheet

bash

Prometheus

`wget https://github.com/prometheus/prometheus/releases/download/v2.45.0/prometheus-2.45.0.linux-amd64.tar.gz`

`tar -xvf prometheus-2.45.0.linux-amd64.tar.gz`

`sudo mv prometheus-2.45.0.linux-amd64 /usr/local/prometheus`

`sudo mv /usr/local/prometheus/prometheus /usr/local/bin/`

`prometheus --config.file=/usr/local/prometheus/prometheus.yml`

Grafana

`sudo apt-get install -y grafana`

`sudo systemctl start grafana-server`

```
sudo systemctl enable grafana-server
sudo systemctl status grafana-server

# EKS Cleanup
eksctl delete cluster --name flask-app-cluster --region us-east-1
aws eks list-clusters --region us-east-1

# EC2 Cleanup
# (Via AWS Console or CLI)
aws ec2 terminate-instances --instance-ids <id1> <id2>

# ECR Cleanup
aws ecr delete-repository --repository-name flask-app --force

# S3 Cleanup
aws s3 rb s3://<bucket-name> --force
```

13. Common Errors & Fixes

Error	Cause	Fix
Grafana data source failed	Wrong URL format	http://<IP>:9090 (no slash)
Prometheus target DOWN	App not accessible	Check security group port 5000
Fleet Request quota	AWS account limit	Contact AWS Support
PVC denied	Storage class issue	Deploy outside EKS
eksctl delete cluster fails	Dependencies exist	Delete manually, then retry
CloudFormation stack stuck	Resource dependency	Wait or force delete

14. Final Project Status

text

PROJECT COMPLETE

✔

- ✓ Experiments + MLflow tracking
- ✓ End-to-end ML pipeline
- ✓ DVC automation + S3 versioning
- ✓ Model Registry (auto-registration)
- ✓ Flask app + model serving
- ✓ Prometheus metrics exposed
- ✓ CI/CD pipeline (GitHub Actions)
- ✓ Model testing + auto-promotion
- ✓ Docker image build + push to ECR
- ✓ EKS cluster + deployment
- ✓ App accessible via LoadBalancer URL
- ✓ Prometheus + Grafana monitoring
- ✓ AWS resources cleaned up

15. Final Takeaways

#	Takeaway
1	Prometheus scrapes metrics from app every 15s
2	Grafana visualizes Prometheus data
3	Separate EC2 for monitoring when PVC fails
4	CloudFormation manages EKS infrastructure
5	Fleet Request quota can block cluster creation
6	PVC errors are common in EKS monitoring
7	AWS cleanup is critical — delete everything
8	Contact AWS Support after cleanup
9	Billing alarms prevent surprise charges
10	End-to-end MLOps = data → model → deploy → monitor
11	5% rule — completing this puts you in top 5%

#	Takeaway
12	Debugging is normal — zoom out, don't give up

16. What's Next?

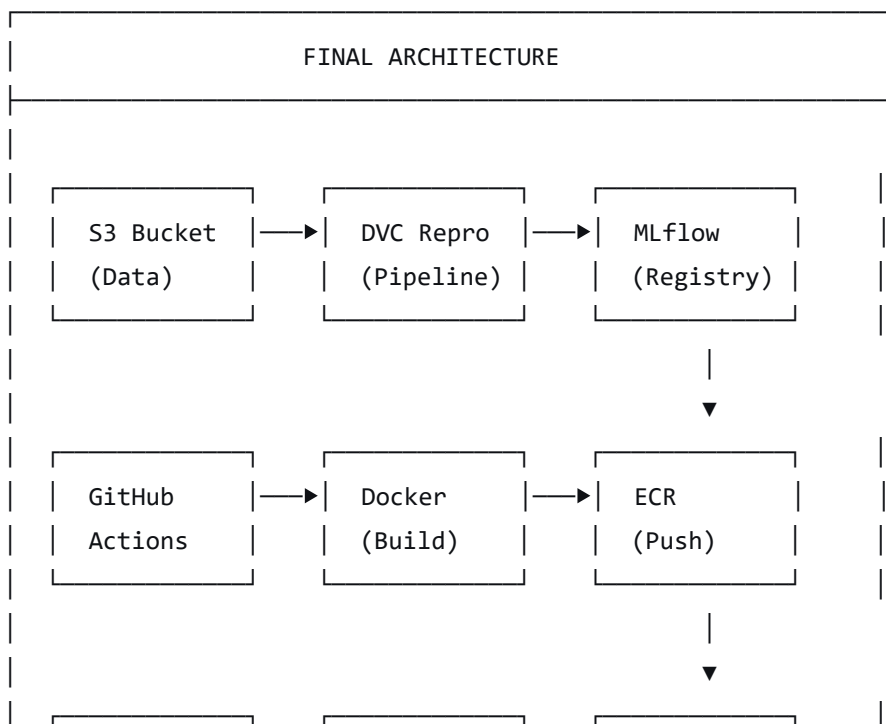
Instructor's Note: "This completes the capstone project series. You've built a fully deployed, production-grade MLOps project on AWS EKS with monitoring. Next up in the playlist: more advanced topics like LLMOps, Gen AI, and agentic AI."

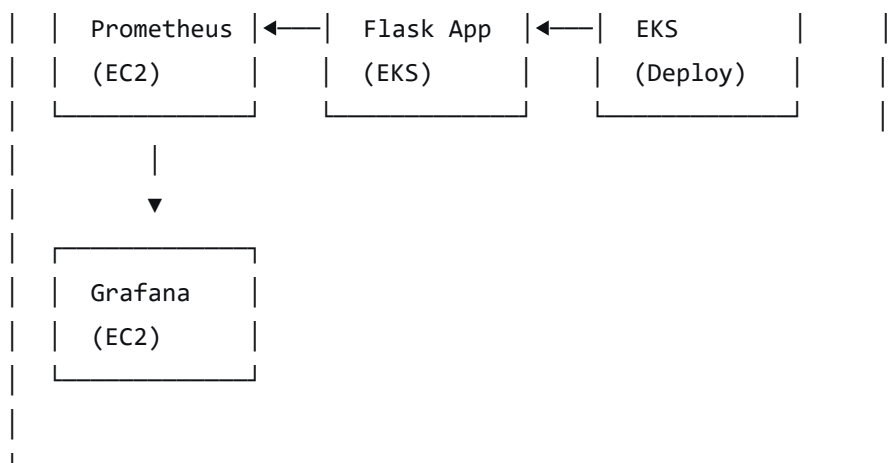
Future content:

- LLMOps (LLM operations)
- Gen AI deployment pipelines
- Agentic AI systems
- Azure/GCP MLOps
- More capstone projects

17. Complete Architecture (Final)

text





18. Congratulations! 🎉

You've completed the **end-to-end MLOps capstone project**:

- ✓ Data versioning with DVC + S3
- ✓ Experiment tracking with MLflow
- ✓ Model registry + auto-promotion
- ✓ CI/CD with GitHub Actions
- ✓ Docker + ECR
- ✓ EKS deployment
- ✓ Prometheus + Grafana monitoring
- ✓ AWS cleanup

You are now in the top 5% of MLOps learners.

📌 End of Notes — Part 6 (Final)

give me interview question with answer of all the things covered in this chat topicwise

🎯 MLOps Capstone Project — Complete Interview Q&A Guide

Based on everything covered across all 6 parts, here are **topic-wise interview questions with detailed answers**.

TOPIC 1: MLOps Fundamentals

Q1: What is MLOps and why is it important?

Answer:

MLOps (Machine Learning Operations) is the practice of combining ML, DevOps, and data engineering to deploy, monitor, and maintain ML models in production reliably and efficiently.

Why important:

- Bridges gap between ML experiments and production
- Automates the ML lifecycle (data → train → deploy → monitor)
- Ensures reproducibility, scalability, and reliability
- Enables continuous retraining and model versioning
- Reduces time-to-production for models

Key components:

- Data versioning
- Experiment tracking
- Model registry
- CI/CD pipelines
- Monitoring & alerting

Q2: What is the difference between a traditional ML project and an MLOps project?

Answer:

Aspect	Traditional ML	MLOps
Code	Notebook-based	Modular, production-grade
Data	Static CSV	Versioned (DVC + S3)

Aspect	Traditional ML	MLOps
Experiments	Manual tracking	MLflow tracking
Model	Manual save/load	Model Registry
Deployment	Local script	Docker + K8s (EKS)
Monitoring	None	Prometheus + Grafana
Retraining	Manual	Automated pipeline
CI/CD	None	GitHub Actions

Q3: Explain the end-to-end MLOps lifecycle.

Answer:

text

1. Data Ingestion (S3 → local)
2. Data Preprocessing (text cleaning)
3. Feature Engineering (Bag of Words)
4. Model Training (Logistic Regression)
5. Model Evaluation (metrics + MLflow)
6. Model Registration (MLflow Registry)
7. Model Serving (Flask app)
8. CI/CD (GitHub Actions)
9. Containerization (Docker)
10. Deployment (EKS)
11. Monitoring (Prometheus + Grafana)
12. Retraining (DVC pipeline)



TOPIC 2: Cookiecutter & Project Structure

Q4: What is Cookiecutter and why use it?

Answer:

Cookiecutter is a tool that generates project skeletons from templates.

Why use it:

- Standardized folder structure
- Reduces manual setup errors
- Saves time on boilerplate
- Follows industry best practices

Usage:

```
bash
```

```
pip install cookiecutter  
cookiecutter -c v1 https://github.com/drivendata/cookiecutter-data-science
```

Generated structure:

```
text
```

```
data/      → raw, interim, processed  
models/    → trained models  
notebooks/ → experiments  
src/       → source code  
reports/   → metrics, figures
```

Q5: What is the difference between `src/models` and `models/` folder?

Answer:

- `src/models/` → **code** (`model_building.py`, `model_evaluation.py`)
- `models/` → **artifacts** (`model.pkl`, `vectorizer.pkl`)

Conflict: Cookiecutter creates both with same name → causes confusion.

Fix: Rename `src/models/` → `src/model/`



TOPIC 3: Experiment Tracking (MLflow + DagsHub)

Q6: What is MLflow and its components?

Answer:

MLflow is an open-source platform for the ML lifecycle.

4 components:

1. **Tracking** — log params, metrics, artifacts
2. **Projects** — package code for reproducibility
3. **Models** — standard model format
4. **Registry** — central model store with versioning

Example:

```
python

import mlflow

with mlflow.start_run():
    mlflow.log_param("C", 10)
    mlflow.log_metric("accuracy", 0.85)
    mlflow.sklearn.log_model(model, "model")
```

Q7: What is DagsHub and why use it instead of local MLflow?**Answer:**

DagsHub is a **remote MLflow tracking server** hosted on the cloud.

Advantages over local MLflow:

Local MLflow	DagsHub
Runs on localhost	Cloud-hosted
Lost on system restart	Persistent
Not shareable	Shareable via URL
No team collaboration	Team access

Setup:

```
python
```

```
os.environ["MLFLOW_TRACKING_URI"] = "https://dagshub.com/<user>/<repo>.mlflow"
os.environ["MLFLOW_TRACKING_USERNAME"] = "<username>"
os.environ["MLFLOW_TRACKING_PASSWORD"] = "<token>"
```

Q8: What are nested runs in MLflow?

Answer:

Nested runs organize **related experiments** under a parent run.

Use case: Hyperparameter tuning — parent = tuning session, children = individual combos.

```
python
```

```
with mlflow.start_run(run_name="LR Hyperparameter Tuning"): # Parent
    for C in [0.1, 1, 10]:
        for penalty in ["l1", "l2"]:
            with mlflow.start_run(nested=True): # Child
                mlflow.log_param("C", C)
                mlflow.log_param("penalty", penalty)
```

Benefit: Compare all combinations in one view.

Q9: What is the difference between log_param , log_metric , and log_artifact ?

Answer:

Method	Purpose	Example
log_param	Hyperparameters	C=10 , max_features=50
log_metric	Performance metrics	accuracy=0.85
log_artifact	Files	metrics.json , plots
log_model	Model object	sklearn model

TOPIC 4: Data Versioning (DVC)

Q10: What is DVC and why is it needed?

Answer:

DVC (Data Version Control) is **Git for data and models**.

Why needed:

- Git can't handle large files (data, models)
- DVC tracks large files via **MD5 hashes**
- Stores actual data in remote storage (S3)
- Enables reproducibility

Commands:

```
bash
```

```
dvc init  
dvc add data/  
dvc push  
dvc pull  
dvc repro
```

Q11: What is the difference between `dvc add` and `dvc repro` ?

Answer:

<code>dvc add</code>	<code>dvc repro</code>
Tracks a specific file/folder	Runs the entire pipeline
Manual step	Automated from <code>dvc.yaml</code>
Creates <code>.dvc</code> file	Creates <code>.dvc/lock</code>
For raw data	For pipeline stages

Q12: Explain dvc.yaml structure.

Answer:

```
yaml

stages:
  data_ingestion:
    cmd: python src/data/data_ingestion.py
    deps:
      - src/data/data_ingestion.py
    outs:
      - data/raw/train.csv
      - data/raw/test.csv
    params:
      - data_ingestion.test_size
```

Key	Meaning
cmd	Command to run
deps	Input dependencies
outs	Output artifacts
params	Values from params.yaml

Run: dvc repro

Q13: How does DVC detect changes?

Answer:

DVC uses **MD5 hashes** stored in .dvc/lock :

- Hash of code (script)
- Hash of dependencies (inputs)
- Hash of outputs

Flow:

```
bash
```

```
# Change params
echo "test_size: 0.25" >> params.yaml

# DVC detects change
dvc status
# → data_ingestion changed

# Re-run only affected stages
dvc repro
```

Q14: What is `params.yaml` and why use it?

Answer:

`params.yaml` centralizes all hyperparameters → no hardcoding.

```
yaml

data_ingestion:
  test_size: 0.2
feature_engineering:
  max_features: 50
```

Usage:

```
python

import yaml
with open("params.yaml") as f:
    params = yaml.safe_load(f)
test_size = params["data_ingestion"]["test_size"]
```

Benefit: Change params → DVC re-runs only affected stages.

TOPIC 5: ML Pipeline Components

Q15: What is the role of a logger module in MLOps?

Answer:

Centralized logging for debugging and monitoring.

python

```
import logging

logging.basicConfig(
    level=logging.DEBUG,
    format="[%(asctime)s] %(lineno)d %(name)s - %(levelname)s - %(message)s",
    handlers=[
        logging.FileHandler("logs/app.log"),
        logging.StreamHandler()
    ]
)

logger = logging.getLogger("mlops")
```

Benefits:

- Track pipeline execution
- Debug failures
- Production monitoring

Q16: What is data ingestion and how is S3 used?**Answer:**

Data ingestion fetches raw data from a source (S3) into the pipeline.

python

```
import boto3

class S3Operation:
    def __init__(self, bucket_name, access_key, secret_key):
        self.s3_client = boto3.client(
            "s3",
            aws_access_key_id=access_key,
            aws_secret_access_key=secret_key
        )

    def fetch_file_from_s3(self, file_name):
        obj = self.s3_client.get_object(Bucket=self.bucket_name, Key=file_name)
```

```
df = pd.read_csv(obj["Body"])  
return df
```

Why S3?

- Industry-standard data source
- Scalable, durable
- Integrates with DVC

Q17: Why is `fit_transform` used on train and `transform` on test?

Answer:

- `fit_transform(train)` → learns vocabulary + transforms
- `transform(test)` → only transforms using learned vocabulary

If you `fit_transform` on test:

- Vectorizer learns test patterns → **data leakage**
- Unrealistic performance metrics

python

```
X_train_bow = vectorizer.fit_transform(X_train)  # Learn + transform  
X_test_bow  = vectorizer.transform(X_test)       # Only transform
```

Q18: Why save the vectorizer separately?

Answer:

During inference, raw text must be transformed into the **same vector space** as training.

python

```
# Save during training  
with open("models/vectorizer.pkl", "wb") as f:  
    pickle.dump(vectorizer, f)  
  
# Load during inference  
with open("models/vectorizer.pkl", "rb") as f:
```

```
vectorizer = pickle.load(f)
features = vectorizer.transform([cleaned_text]).toarray()
```

If not saved: Must re-fit → different vocabulary → wrong predictions.

TOPIC 6: Model Registry & Serving

Q19: What is MLflow Model Registry?

Answer:

Centralized hub for ML models with versioning and stage transitions.

Stages:

text

None → Staging → Production → Archived

Stage	Purpose
Staging	Newly trained, awaiting validation
Production	Live model serving traffic
Archived	Deprecated models

Register model:

python

```
model_uri = f"runs:{run_id}/model"
mlflow.register_model(model_uri, "my_model")
```

Q20: What is model serving and how is it implemented?

Answer:

Model serving = deploying a model as an API for predictions.

Dynamic serving from Registry:

python

```
def get_latest_model_version(model_name, stage="Production"):
    client = mlflow.tracking.MlflowClient()
    latest = client.get_latest_versions(model_name, stages=[stage])
    return latest[0].version

model_uri = f"models:{MODEL_NAME}/{MODEL_VERSION}"
model = mlflow.sklearn.load_model(model_uri)
```

Benefit: Always serves latest production model — no manual swap.

Q21: What is model retraining and how is it automated?

Answer:

Model retraining = automatically retraining on new data.

Automation flow:

text

1. New data arrives in S3
2. dvc repro → re-runs pipeline
3. New model trained → evaluated
4. MLflow registers new version (v2, v3...)
5. New version → Staging
6. Tests pass → Production

Trigger options:

- Scheduled (cron)
- Data drift detected
- Performance drop

TOPIC 7: CI/CD (GitHub Actions)

Q22: What is CI/CD in MLOps?

Answer:

- **CI (Continuous Integration):** Automatically test code changes
- **CD (Continuous Deployment):** Automatically deploy passing changes

MLOps CI/CD stages:

text

1. Checkout code
2. Setup Python
3. Install dependencies
4. Run pipeline (dvc repro)
5. Run model tests
6. Promote model to production
7. Run app tests
8. Build Docker image
9. Push to ECR
10. Deploy to EKS

Q23: How do you handle secrets in CI/CD?

Answer:

Use **GitHub Secrets** — never hardcode credentials.

text

Settings → Secrets and variables → Actions → New repository secret

Secrets used:

Secret	Purpose
CAPSTONE_TEST	DagsHub token
AWS_ACCESS_KEY_ID	IAM access key
AWS_SECRET_ACCESS_KEY	IAM secret key
AWS_DEFAULT_REGION	us-east-1

Secret	Purpose
ECR_REPOSITORY	flask-app
AWS_ACCOUNT_ID	AWS account ID

Usage:

```
yaml

env:
  CAPSTONE_TEST: ${ secrets.CAPSTONE_TEST }
```

Q24: What is the difference between CI and CD in this project?

Answer:

CI	CD
Checkout, setup, install	Build Docker image
Run pipeline (dvc repro)	Push to ECR
Run model tests	Deploy to EKS
Run Flask app tests	Update K8s deployment
Promote model	Expose via LoadBalancer

 TOPIC 8: Docker & ECR

Q25: Why use Docker in MLOps?

Answer:

Docker packages app + dependencies into a **portable container**.

Benefits:

- Consistent environment (dev = prod)
- Isolated dependencies
- Easy deployment to any cloud
- Version-controlled images

Dockerfile:

```
dockerfile

FROM python:3.10-slim
WORKDIR /app
COPY flask_app/ /app/
COPY models/vectorizer.pkl /app/models/
RUN pip install --no-cache-dir -r requirements.txt
EXPOSE 5000
CMD ["python", "app.py"]
```

Q26: Why use a separate requirements.txt for the Flask app?

Answer:

- Main requirements.txt = DVC, MLflow, sklearn, etc. (~2GB image)
- Flask app requirements.txt = Flask, mlflow.sklearn, prometheus_client (~500MB)

Generate optimized:

```
bash

pip install pipreqs
cd flask_app
pipreqs . --force
```

Benefit: Smaller image → faster builds, faster deployments.

Q27: What is ECR and why not Docker Hub?

Answer:

ECR (Elastic Container Registry) is AWS's Docker registry.

Feature	ECR	Docker Hub
Provider	AWS	Docker
Auth	IAM	Docker account
Integration	Native AWS	Manual
Private repos	Free (500MB)	Paid
Best for	AWS deployments	Public images

Q28: How do you handle environment variables in Docker?

Answer:

Pass at runtime:

```
bash
```

```
docker run -p 5000:5000 \  
-e CAPSTONE_TEST=<token> \  
flask-app
```

In code:

```
python
```

```
os.environ["MLFLOW_TRACKING_PASSWORD"] = os.getenv("CAPSTONE_TEST")
```

Why: Never bake secrets into image.

TOPIC 9: Kubernetes & EKS

Q29: What is EKS and why use it?

Answer:

EKS (Elastic Kubernetes Service) is AWS's **managed Kubernetes**.

Benefits:

- AWS manages control plane
- Auto-scaling, health checks
- Integrates with AWS services (IAM, VPC, ELB)
- Production-grade orchestration

Create cluster:

```
bash
```

```
eksctl create cluster \  
  --name flask-app-cluster \  
  --region us-east-1 \  
  --nodegroup-name flask-app-nodes \  
  --node-type t3.small \  
  --nodes 1 \  
  --managed
```

Q30: What is the difference between Deployment and Service in Kubernetes?

Answer:

Deployment	Service
Manages pod replicas	Exposes pods via network
Handles rolling updates	Provides stable endpoint
kind: Deployment	kind: Service
Ensures desired state	Load balances traffic

Deployment:

```
yaml
```

```
kind: Deployment  
spec:  
  replicas: 2
```

```
template:
  spec:
    containers:
      - name: flask-app
        image: <ECR-URL>:latest
```

Service:

yaml

```
kind: Service
spec:
  type: LoadBalancer
  ports:
    - port: 5000
```

Q31: What is ImagePullBackOff and how to fix it?

Answer:

Kubernetes can't pull the Docker image.

Causes:

- Wrong image name/tag
- ECR authentication failed
- Image doesn't exist
- Network issue

Fix:

bash

```
kubectl describe pod <pod-name> # Check events
kubectl logs <pod-name>         # Check Logs
```

Instructor's case: ECR repo name mismatch (flask-app VS capstone-app).

Q32: How do you pass secrets to Kubernetes pods?

Answer:**Use Kubernetes Secrets:**

bash

```
kubectl create secret generic capstone-test \
  --from-literal=CAPSTONE_TEST=<token>
```

Reference in deployment:

yaml

```
env:
  - name: CAPSTONE_TEST
    valueFrom:
      secretKeyRef:
        name: capstone-test
        key: CAPSTONE_TEST
```

Q33: What is a LoadBalancer service?**Answer:**

Provisions an **AWS ELB** to expose pods externally.

yaml

```
kind: Service
spec:
  type: LoadBalancer
  ports:
    - port: 5000
      targetPort: 5000
```

Flow:

text

Internet → ELB (public DNS) → Service → Pod (internal IP)

Get URL:

```
bash
```

```
kubectl get svc flask-app-service  
# EXTERNAL-IP: a1b2c3.elb.amazonaws.com
```

Q34: What is CloudFormation and how does EKS use it?

Answer:

CloudFormation = AWS's **Infrastructure as Code** service.

How EKS uses it:

```
text
```

```
eksctl create cluster  
↓  
CloudFormation Stack 1: EKS control plane  
CloudFormation Stack 2: Node group (EC2, VPC, subnets)
```

Key concepts:

Term	Meaning
Stack	Logical grouping of resources
Template	JSON/YAML defining resources
Provisioning	Creating resources from template

Benefit: Auto-recreates failed resources.



TOPIC 10: Monitoring (Prometheus + Grafana)

Q35: What is Prometheus and how does it work?

Answer:

Prometheus is an **open-source monitoring system** that scrapes metrics.

Pull model:

text

Prometheus → HTTP GET /metrics → Target

↑

└─ Every 15 seconds

Config:

yaml

```
global:
  scrape_interval: 15s
scrape_configs:
  - job_name: "flask-app"
    static_configs:
      - targets: ["<app-url>:5000"]
```

Metrics types:

Type	Use Case
Counter	Total requests
Gauge	Current memory
Histogram	Latency distribution
Summary	Quantiles

Q36: What metrics did we expose in the Flask app?

Answer:

python

```
from prometheus_client import Counter, Histogram

# Total requests
REQUEST_COUNT = Counter(
    "app_request_count", "Total requests",
```

```
    ["method", "endpoint", "http_status"]
)

# Request Latency
REQUEST_LATENCY = Histogram(
    "app_request_latency_seconds", "Latency",
    ["endpoint"]
)

# Predictions per class
PREDICTION_COUNT = Counter(
    "app_prediction_count", "Predictions",
    ["prediction_class"]
)
```

Why these?

- Request count → traffic volume
- Latency → performance monitoring
- Prediction count → model behavior, drift

Q37: What is Grafana and how does it connect to Prometheus?

Answer:

Grafana is a **visualization platform** for metrics.

Connection:

text

```
Grafana → Connections → Data Sources → Add Prometheus
→ URL: http://<prometheus-ip>:9090
→ Save & Test
```

Build dashboard:

text

```
New Dashboard → Add Visualization
→ Query: app_request_count_total
→ Panel title: "Request to Home Route"
```

Q38: Why did Prometheus/Grafana fail inside EKS?

Answer:

PVC (Persistent Volume Claim) error.

What is PVC?

- Request for storage by K8s app
- Bound to PV (Persistent Volume)
- Provisioned by Storage Class

Error: PVC request denied → storage couldn't be provisioned.

Solution: Deploy on **separate EC2 instances**.

Why this is common:

- EKS default storage class may not be configured
- EBS CSI driver may not be installed
- Industry practice: monitoring on separate servers



TOPIC 11: AWS Cleanup & Cost Management

Q39: How do you clean up AWS resources?

Answer:

```
bash
```

```
# 1. Terminate EC2 instances
```

```
aws ec2 terminate-instances --instance-ids <id1> <id2>
```

```
# 2. Delete EKS cluster
```

```
eksctl delete cluster --name flask-app-cluster --region us-east-1
```

```
# 3. Delete ECR repository
```

```
aws ecr delete-repository --repository-name flask-app --force
```

```
# 4. Delete S3 bucket (optional)
```

```
aws s3 rb s3://<bucket-name> --force
```

5. Verify

```
aws eks list-clusters --region us-east-1
```

```
aws ec2 describe-instances --filters "Name=instance-state-name,Values=running"
```

Then: Contact AWS Support to confirm all chargeable services are terminated.

Q40: What is a Fleet Request quota issue?

Answer:

Fleet Request = AWS internal process for provisioning EC2 instances.

Flow:

text

```
eksctl create cluster → Auto Scaling Group → Fleet Request → EC2 instances
```

Error: "You have reached your quota for maximum Fleet Requests"

Cause: AWS account quota limits

Fix:

1. Contact AWS Support via chat
2. Request quota increase
3. Explain it's for learning/development



TOPIC 12: Scenario-Based Questions

Q41: A model performs well in development but poorly in production. What could be wrong?

Answer:

Possible causes:

1. **Data leakage** — `fit_transform` on test data
2. **Preprocessing mismatch** — different cleaning in train vs serve

3. **Vectorizer not saved** — re-fit on inference data
4. **Data drift** — production data differs from training
5. **Feature mismatch** — wrong column order
6. **Model version mismatch** — serving wrong model

Debugging:

- Check preprocessing consistency
- Verify vectorizer loaded correctly
- Compare train vs production distributions
- Check MLflow registry stage

Q42: Your CI/CD pipeline fails at `dvc repro`. How do you debug?

Answer:

Steps:

1. Check GitHub Actions logs
2. Look for missing files (`FileNotFoundError`)
3. Verify `src/` is not in `.gitignore`
4. Check DVC remote credentials
5. Run `dvc repro` locally to reproduce
6. Check `dvc status` for changes

Common issues:

- `src/` in `.gitignore` → not pushed
- `data/` tracked by Git instead of DVC
- Missing AWS credentials

Instructor's case: `src/data/` was in `.gitignore` → fixed by removing.

Q43: How would you handle model drift in production?

Answer:**Detection:**

- Monitor prediction distribution (Prometheus)
- Compare with training distribution
- Statistical tests (KS test, PSI)

Response:

1. **Alert** (Grafana alert)
2. **Trigger retraining** (DVC pipeline)
3. **Evaluate new model** (tests)
4. **Auto-promote** if better (MLflow Registry)
5. **Deploy** new version (CI/CD)

Tools:

- Prometheus for metrics
- Grafana for visualization
- MLflow for registry
- DVC for pipeline

Q44: How do you ensure reproducibility in MLOps?**Answer:**

Layer	Tool	Purpose
Code	Git	Version control
Data	DVC	Data versioning
Environment	Docker	Consistent dependencies
Experiments	MLflow	Track params, metrics
Models	MLflow Registry	Model versioning

Layer	Tool	Purpose
Pipeline	DVC	Reproducible stages

Key: Every artifact has a hash → can recreate any version.

Q45: What is the difference between a Counter and a Histogram in Prometheus?

Answer:

Counter	Histogram
Monotonically increasing	Bucketed distribution
<code>inc()</code> only	<code>observe(value)</code>
Total requests	Request latency
Can't decrease	Tracks quantiles

Example:

```
python

# Counter
REQUEST_COUNT.inc() # +1

# Histogram
REQUEST_LATENCY.observe(0.5) # record 500ms
```

Use case:

- Counter → "How many requests?"
- Histogram → "How long did they take?"

TOPIC 13: Advanced Concepts

Q46: What is the difference between `dvc add` and `dvc commit` ?

Answer:

<code>dvc add</code>	<code>dvc commit</code>
Tracks a new file/folder	Commits pipeline outputs
Creates <code>.dvc</code> file	Updates <code>.dvc/lock</code>
For raw data	After <code>dvc repro</code>

Flow:

```
bash
```

```
dvc add data/      # Track raw data
dvc repro          # Run pipeline
dvc commit         # Commit outputs
dvc push           # Push to remote
```

Q47: What is the difference between `mlflow.log_model` and `mlflow.register_model` ?

Answer:

<code>log_model</code>	<code>register_model</code>
Saves model to run artifacts	Adds model to Registry
Per-run	Per-model-name
Not versioned	Versioned (v1, v2...)
For tracking	For serving

Flow:

```
python
```

```
mlflow.sklearn.log_model(model, "model") # Log
model_uri = f"runs:{run_id}/model"
mlflow.register_model(model_uri, "my_model") # Register
```

Q48: What is the role of experiment_info.json ?

Answer:

Bridge between **evaluation** and **registration** stages.

json

```
{
  "run_id": "abc123",
  "model_path": "model"
}
```

Why needed:

- Registration needs `run_id` to construct model URI
- Passed between DVC stages
- Decouples evaluation from registration

Q49: How does MLflow Model Registry handle versioning?

Answer:

text

```
Model: my_model
├─ Version 1 (Archived)
├─ Version 2 (Archived)
├─ Version 3 (Production)
└─ Version 4 (Staging)
```

Version transitions:

python

```
client = MlflowClient()
client.transition_model_version_stage(
    name="my_model",
    version=4,
    stage="Production"
)
```

Stages: None → Staging → Production → Archived

Q50: What is the difference between CI and CD in MLOps vs traditional DevOps?

Answer:

Aspect	Traditional DevOps	MLOps
CI tests	Unit, integration	+ Model tests
CD deploys	Code	Code + Model
Artifacts	Binary	Model + Vectorizer
Versioning	Code only	Code + Data + Model
Monitoring	App metrics	+ Model drift

MLOps adds:

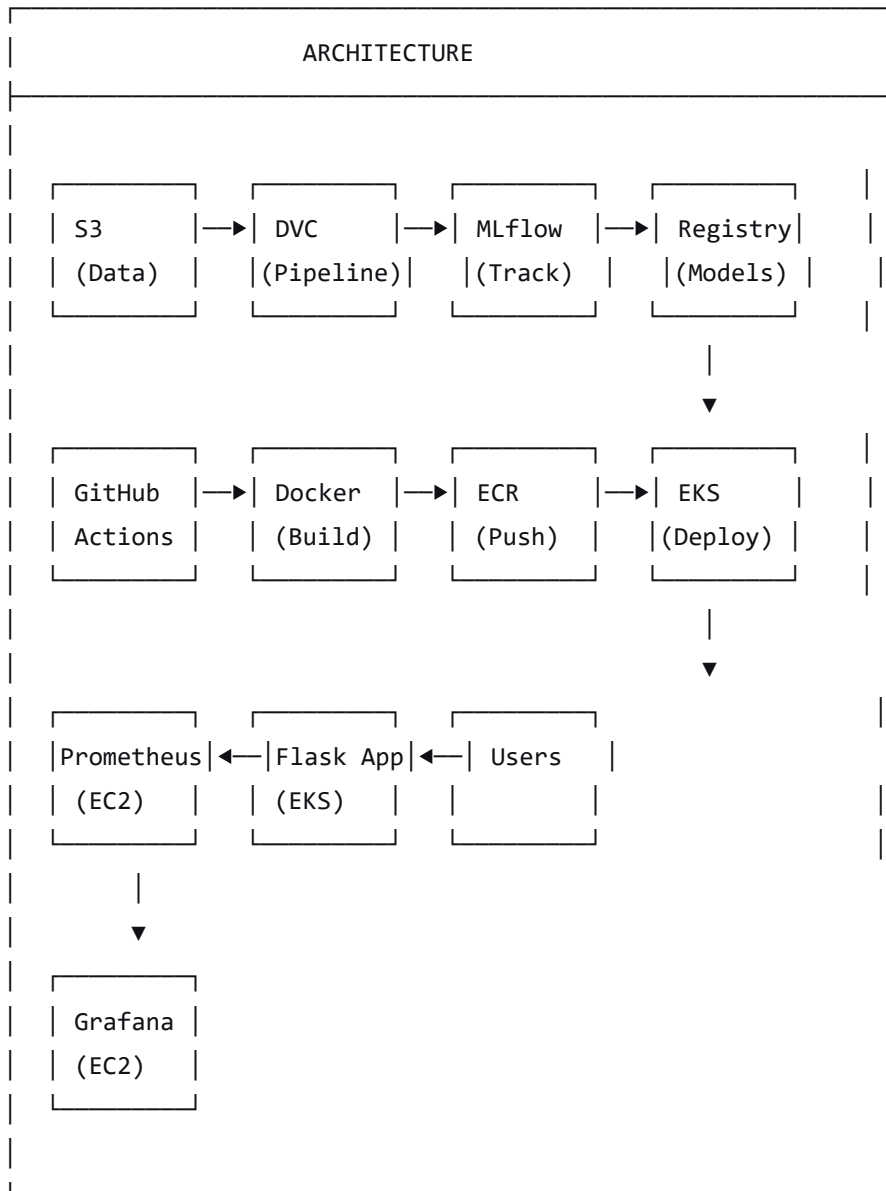
- Model performance tests
- Data versioning
- Model registry
- Drift detection

TOPIC 14: System Design

Q51: Design an MLOps system for a text classification model.

Answer:

text



Components:

1. **Data:** S3 + DVC
2. **Experiments:** MLflow + DagsHub
3. **Pipeline:** DVC stages
4. **CI/CD:** GitHub Actions
5. **Container:** Docker + ECR
6. **Deploy:** EKS
7. **Serve:** Flask + LoadBalancer

8. Monitor: Prometheus + Grafana

Q52: How would you scale this system for 1M requests/day?

Answer:

Scaling strategies:

Layer	Strategy
Ingestion	S3 multipart upload, batch processing
Training	Distributed training (Spark, Ray)
Serving	Horizontal pod autoscaling (HPA)
Load	Application Load Balancer
Cache	Redis for frequent predictions
DB	RDS for metadata
Monitoring	Prometheus federation
Queue	SQS for async predictions

EKS scaling:

```
yaml

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```



TOPIC 15: Behavioral & Project Questions

Q53: Walk me through your MLOps capstone project.

Answer:

Project: IMDB Sentiment Classification (End-to-End MLOps)

Problem: Classify movie reviews as positive/negative.

Architecture:

1. **Data:** S3 → DVC versioned
2. **Experiments:** 3 experiments tracked on DagsHub
3. **Pipeline:** Data ingestion → preprocessing → feature engineering → training → evaluation → registration
4. **Registry:** MLflow with auto-promotion
5. **CI/CD:** GitHub Actions
6. **Deploy:** Docker → ECR → EKS
7. **Monitor:** Prometheus + Grafana

Key achievements:

- Automated retraining pipeline
- Auto model promotion (Staging → Production)
- 360° monitoring
- Full reproducibility

Challenges:

- EKS setup (1-2 weeks)
- PVC errors for Prometheus in EKS
- Fleet Request quota issues
- AWS billing surprises (₹65,000 bill)

Q54: What was the hardest part of your project?

Answer:

EKS deployment was the hardest.

Challenges:

1. **eksctl setup** — dependency conflicts on Windows
2. **Fleet Request quota** — blocked cluster creation
3. **PVC errors** — Prometheus/Grafana failed inside EKS
4. **AWS billing** — ₹65,000 surprise bill (forgotten resources)
5. **ImagePullBackOff** — ECR repo name mismatch

How I overcame:

- Extensive debugging with AWS Support
- Deployed monitoring on separate EC2 instances
- Documented all errors and solutions
- Zoomed out when stuck (root cause was obvious)

Lesson: Production MLOps is 80% debugging, 20% coding.

Q55: What would you do differently?**Answer:**

1. **Set up AWS Budgets** from day 1 (avoid ₹65K bill)
2. **Use separate environments** (dev/staging/prod)
3. **Add more tests** (integration, load testing)
4. **Implement drift detection** (Evidently AI)
5. **Use Terraform** instead of manual AWS setup
6. **Add model explainability** (SHAP, LIME)
7. **Implement A/B testing** for model comparison
8. **Use GitOps** (ArgoCD) for deployment

Q56: How do you stay updated with MLOps?

Answer:

- **Follow:** MLOps Community, Made With ML
- **Read:** Chip Huyen's "Designing ML Systems"
- **Tools:** MLflow, DVC, Kubeflow, BentoML
- **Practice:** Build projects, contribute to OSS
- **Courses:** [DeepLearning.AI](#) MLOps Specialization
- **Newsletters:** MLOps Weekly, The Batch

Q57: Explain the 5% rule in your project.**Answer:****5% Rule:**

text

100% watch intro video

↓

5% reach end of project

↓

5% get CV shortlisted

↓

5% get interview calls

↓

5% get selected/transition

Meaning: Completing an end-to-end project puts you in the top 5% because:

- Most people give up midway
- Few implement everything
- Even fewer debug production issues

Instructor's guarantee: "If you complete this end-to-end, you're in the top 5%."

 TOPIC 16: Quick Fire Questions**Q58: What port does Prometheus run on?**

Answer: 9090

Q59: What port does Grafana run on?

Answer: 3000

Q60: What port does Flask run on?

Answer: 5000

Q61: What is the default MLflow port?

Answer: 5000 (same as Flask — conflict!)

Q62: What does DVC stand for?

Answer: Data Version Control

Q63: What does EKS stand for?

Answer: Elastic Kubernetes Service

Q64: What does ECR stand for?

Answer: Elastic Container Registry

Q65: What does PVC stand for?

Answer: Persistent Volume Claim

Q66: What does HPA stand for?

Answer: Horizontal Pod Autoscaler

Q67: What is the default scrape interval in Prometheus?

Answer: 15 seconds

Q68: What are the MLflow Registry stages?

Answer: None, Staging, Production, Archived

Q69: What is the command to run a DVC pipeline?

Answer: `dvc repro`

Q70: What is the command to create an EKS cluster?

Answer: `eksctl create cluster --name <name> --region <region>`

FINAL TIPS FOR INTERVIEWS

1. **Emphasize end-to-end** — you built the entire pipeline
2. **Mention debugging** — production issues are normal
3. **Talk about trade-offs** — why EKS vs EC2, why S3 vs MongoDB
4. **Show awareness** — AWS costs, security, scaling
5. **Be honest** — "I faced X error, I debugged it by Y"
6. **Connect to business** — model serving, monitoring, retraining
7. **Demonstrate automation** — CI/CD, auto-promotion, DVC

 **End of Interview Q&A Guide — 70 Questions**